

Belief Geometry on the Random Hierarchy Model

A small transformer partially encodes, and causally uses, the ground truth tree posterior, probed against ground truth

simplex-rhm-belief · run of 29 June 2026 · all numbers from the recorded pipeline output

Abstract. We reproduce, in miniature, the Simplex belief-geometry result on a hierarchical generative process. We train a 2-layer GPT-2 decoder ($\approx 399\text{k}$ parameters) by next-token prediction on samples from a Random Hierarchy Model (RHM; arity $s = 2$, depth $L = 3$, vocabulary $v = 8$), a grammar small enough (1024 equiprobable trees) to enumerate exactly. We compute the **ground truth** Bayesian posterior over the tree’s hidden latents by sum-product belief propagation, verified against brute-force enumeration to $< 10^{-6}$. The trained model reaches a test cross-entropy of 0.88 nats, closing $\approx 89\%$ of the gap between the uniform baseline (2.08) and the Bayes-optimal floor (0.73). It has effectively learned the posterior predictor. A single global linear probe partially recovers this posterior from the residual stream: held-out $R^2 = 0.38$ for the root class (versus ≈ 0 for a shuffled control), a noisy affine image, with decodability accumulating across depth ($-0.00 \rightarrow 0.15 \rightarrow 0.38$) and the belief readout **blooming** outward from the prior toward simplex vertices as context accumulates. A latent-level sweep shows the residual encodes the **whole** hierarchy, most strongly the locally-predictive deep latents (R^2 up to 0.66) and least strongly the coarse global root. Causal steering on the layer-1 residual confirms the belief is **used**, not merely decodable: steering toward a wrong latent collapses the true next token’s log-probability ($-0.73 \rightarrow -8.5$). We pre-registered our predictions before seeing results; three of four held, and the one miss (root $R^2 < 0.5$) is informative. Finally, a 30-run sweep over 10 random grammars $\times$ 3 replicate seeds shows all four findings — the inverted level gradient, blooming, layer accumulation, and causal steering — replicate across the RHM family with low replicate variance, and that the published **grammar** = 0 numbers sit inside the family distribution. A further 99-run architecture sweep (width, depth, heads, and training budget, one-axis-at-a-time $\times$ 3 grammars $\times$ 3 seeds) shows the phenomenology is capacity-robust down to a low-width floor: decodability rises with width and depth, the inverted gradient holds at every capacity, heads matter least, and decodability is only weakly correlated with how well the model fit the loss.

not holds

1. Introduction

Natural data is hierarchical: characters compose into words, words into phrases, phrases into meaning. The Random Hierarchy Model (RHM) of Cagnetta et al. (2025) models this with a synthetic grammar: a fixed tree in which each high-level symbol expands, via production rules randomly chosen from pre-set, into a fixed-length string of lower-level symbols, down to observed leaves. Such a grammar defines a finite set of equiprobable trees, each of which generates a leaf string. Given a prefix of observed leaves, predicting the next leaf optimally requires inferring the distribution over the **hidden** latent symbols that generated the observed prefix. The optimal next-token predictor is the Bayesian belief state over those latents.

This allows us to see the RHM as a testbed for the belief geometry studies: will the auto-regressive language model represent in its residual stream, some counterpart of the data generating process. For example, will the ground truth posterior distribution over the data-generating process’s hidden states be represented in the residual stream, will this information be used.

factcheck

Earlier Simplex work Shai et al. (2026) found that the residual stream of a trained transformer carries a linear image of the ground truth posterior (seemingly also known as exact posterior) over the hidden states of a HMM generative process. Will this be the case for the RHM as well? In other words, will the residual stream carry a linear image of the ground truth posterior over the hidden states of the RHM generative process?

We test whether, the belief is (i) linearly decodable, (ii) geometrically organized as a simplex that sharpens with context, (iii) built up additively across layers, and (iv) causally relevant to the model’s output.

fact check

In some sense, RHMs are a more natural testbed than HMMs for these questions, because they are hierarchical and have a tree structure, which is more similar to natural language. This does not mean though they are obviously more complex: unlike HMMs, RHMs by default have no recursive generation and yield fixed-length strings which makes them computationally and analytically simpler.

This paper trains a tiny transformer on such grammars and studies whether the residual stream carries the ground truth posterior, where in the network it lives, what geometry it traces, and whether the model relies on it.

We find that the residual stream carries a **partial** linear image of the ground truth posterior: a single global probe recovers the root class at $R^2 = 0.38$ (against ≈ 0 for a shuffled control) and the deeper, locally-predictive latents more strongly (up to 0.66). This image accumulates across layers ($-0.00 \rightarrow 0.15 \rightarrow 0.38$) and **blooms** outward from the prior toward simplex vertices as context grows, and causal steering confirms the model relies on it rather than merely exposing it. We pre-registered these predictions before training; three of four held, the informative miss being the root’s sub-0.5 recovery. The phenomenology replicates across the RHM family (10-grammar $\times$ 3-seed sweep) and is robust to model capacity (99-run architecture sweep) and one tree level deeper ($L = 4$). Finally, breaking the grammar’s invertibility with **ambiguity** leaves a genuinely uncertain belief state, and there the probe does not degrade but **sharpens** — evidence the residual tracks the belief itself, not a memorized leaf $\rightarrow$ root lookup.

2. Experimental design

Methodologically, this paper sits between the two reference points in the bibliography: Cagnetta et al. (2025)’s RHM study uses the same hierarchical data model but evaluates the last-token prediction setting, while Shai et al. (2026)’s **Transformers learn factored representations** motivates pooling predictive vectors across contexts.

The study proceeds in five passes. The **initial** experiment fixes one grammar and one training run and establishes the four core findings — linear decodability, layer accumulation, blooming, and causal use. Three **robustness** passes then test whether these are properties of the RHM family rather than of one draw: a grammar sweep (10 grammars $\times$ 3 seeds, 30 runs), an architecture sweep (11 capacity configs $\times$ 3 grammars $\times$ 3 seeds, 99 runs), and a depth-4 extension (10 grammars $\times$ 2 model depths $\times$ 3 seeds, 60 runs). A final **supplementary** pass generalizes the grammar with two knobs — ambiguity and skew (27 runs) — to engineer a belief state that stays uncertain even at full context, and asks whether the linear image survives. Every posterior used as a probe target is the exact ground truth, computed by the belief propagation derived in Section 2.1 and verified against brute-force enumeration.

2.1. The Random Hierarchy Model and exact beliefs

Grammar. The default parameters of RHMs used are: arity $s = 2$, depth $L = 3$, so each string has $d = s^L = 8$ leaves; per-level vocabulary $v = 8$; and $m = 2$ production rules per parent symbol, chosen uniformly. Rules are drawn once and held fixed, and are unambiguous.

A sample is generated top-down: pick a root class uniformly, recursively expand each symbol by a uniformly chosen rule, and emit the leaf string. Thus, each sample is both the string and the respective parse tree formed of latents that generated this string. With these parameters the grammar admits exactly $v \cdot m^{d-1} = 8 \cdot 2^7 = 1024$ equiprobable distinct trees.

The exact frozen grammar used in this first experiment is shown below (Table 1). See this grammar visualized as a tree in Appendix 3. Symbols are written as S1, ..., S8 for readability (the saved arrays use zero-based ids); at each level the parent takes one of the two listed child pairs uniformly. Symbols are numbered to distinguish them, and their numbering is unrelated to the order of appearance in the derived string. The first symbol of the sampled string would be the one produced by the leftmost branching of the respective sampled parse tree.

Parent	Top expansion	Middle expansion	Bottom expansion
S1	[S3, S1] or [S4, S8]	[S2, S1] or [S1, S5]	[S4, S1] or [S5, S3]
S2	[S4, S3] or [S1, S3]	[S6, S3] or [S3, S1]	[S6, S7] or [S8, S8]

explain why this experiment was actually done, why they fixed grammar first, and what hypothesis does this answer?

Parent	Top expansion	Middle expansion	Bottom expansion
S3	[S6, S7] or [S5, S6]	[S5, S2] or [S1, S1]	[S3, S4] or [S7, S8]
S4	[S6, S8] or [S7, S7]	[S6, S5] or [S8, S6]	[S6, S5] or [S4, S7]
S5	[S6, S2] or [S1, S5]	[S4, S3] or [S1, S8]	[S1, S5] or [S6, S4]
S6	[S8, S6] or [S2, S7]	[S7, S6] or [S4, S5]	[S8, S4] or [S1, S8]
S7	[S4, S7] or [S1, S1]	[S3, S8] or [S4, S1]	[S4, S6] or [S3, S2]
S8	[S2, S2] or [S5, S7]	[S1, S2] or [S8, S3]	[S5, S7] or [S3, S6]

Table 1: Frozen level-specific grammar sampled once before training. Top, middle, and bottom columns correspond to `rules_0`, `rules_1`, and `rules_2` in `artifacts/grammar.npz` (see visualized in appendix Appendix 3).

Naive marginalisation (the reference). Counting the proportion of prefix-consistent trees in which the latent takes a given value yields the exact ground-truth posterior.¹ A tree is defined by its root symbol together with one rule choice at each of the $d - 1$ internal nodes, so we only have $v m^{d-1} = 1024$ trees in our default setup with small trees, making such counting-based approach feasible. This let us verify the belief-propagation posterior derived below against brute-force enumeration values, the two agree to $< 10^{-6}$ across 10 passing tests.

Derivation. Write $z_{\ell,i}$ for the latent symbol at node i of level ℓ (the root is $z_{0,0}$), and $x_1, \dots, x_d$ for the leaves. The grammar defines the joint law: the root is drawn from the uniform prior $\pi(a) = 1/v$, and each node of level ℓ carrying symbol a (in our case, one of the S1, ..., S8 symbols) expands by rule $r \in \{1, \dots, m\}$ with probability $p_\ell(a, r)$ (canonically $1/m$), emitting the child tuple $\rho_\ell(a, r) \in \{1, \dots, v\}^s$. Given a prefix $x_{1:k}$ we want the posterior over any single latent, marginalising the $d - k$ unobserved leaves and every rule choice. The tree is a factor graph without cycles, so sum-product belief propagation returns the exact marginals in one upward and one downward sweep.

The upward pass sends, from each node, the likelihood of the observed leaves in its subtree given the node’s symbol,

$$\mu_{\ell,i}(a) = P(\text{observed leaves under node } (\ell, i) \mid z_{\ell,i} = a).$$

Leaves are the base case: a pinned indicator when observed, an all-ones message when marginalised,

$$\mu_{L,j}(a) = \begin{cases} \mathbb{1}[a = x_j] & \text{if } j \leq k \\ 1 & \text{if } j > k \end{cases}$$

and each internal node combines its children by summing over its own rules,

$$\mu_{\ell,i}(a) = \sum_{r=1}^m p_\ell(a, r) \prod_{j=1}^s \mu_{\ell+1, s(i-1)+j}(\rho_\ell(a, r)_j).$$

The **root** posterior is then the upward message at the top, reweighted by the prior and normalised,²

$$P(z_{0,0} = a \mid x_{1:k}) = \frac{\pi(a) \mu_{0,0}(a)}{\sum_{a'} \pi(a') \mu_{0,0}(a')}.$$

Any **non-root** latent needs, in addition, the evidence from the rest of the tree (co-emitted siblings are correlated via shared parent), carried by a downward message $\lambda_{\ell,i}(a)$ seeded at the root by the prior, $\lambda_{0,0}(a) = \pi(a)$, and pushed to child j (global index $c_j = s(i - 1) + j$) as

$$\lambda_{\ell+1,c_j}(b) = \sum_a \lambda_{\ell,i}(a) \sum_{r: \rho_\ell(a,r)_j=b} p_\ell(a, r) \prod_{j' \neq j} \mu_{\ell+1,c_{j'}}(\rho_\ell(a, r)_{j'}).$$

The marginal at an arbitrary node is the product of the two messages, normalised,³

$$P(z_{\ell,i} = a \mid x_{1:k}) = \frac{\mu_{\ell,i}(a) \lambda_{\ell,i}(a)}{\sum_{a'} \mu_{\ell,i}(a') \lambda_{\ell,i}(a')}.$$

¹when trees are not equiprobable, the count becomes a probability-weighted sum

²In `rhmm.py` terms, this is `belief_root: mu[(0,0)]` times the uniform prior, normalised

³In `rhmm.py` terms, this is `belief_node`

going over a' here
is a denominator
going through all
possible root sym-
bols

Under unambiguity (C2) each child tuple names its parent uniquely, so at full context $k = d$ every upward message collapses to a single symbol and one root survives (entropy 0), the sharpening previewed above. The aforescribed computation is linear against tree size, making it much faster than the naive marginalisation algorithm.

Bayes-optimal floor and uniform baseline. The *optimal* next-token predictor is itself a belief marginal: predicting x_{k+1} from $x_{1:k}$ is the posterior over the leaf node $(L, k + 1)$ left unobserved, obtained from the same two sweeps,

$$P(x_{k+1} = c \mid x_{1:k}) = \frac{\mu_{L,k+1}(c) \lambda_{L,k+1}(c)}{\sum_{c'} \mu_{L,k+1}(c') \lambda_{L,k+1}(c')}.$$

Autoregressive models are judged by their next-token cross-entropy. The lowest attainable mean next-token cross-entropy thus equals the conditional entropy of each next leaf under the true grammar, averaged over prefixes,

$$H_k = H(X_{k+1} \mid X_{1:k}) = \sum_{x_{1:k}} P(x_{1:k}) \left[- \sum_c P(c \mid x_{1:k}) \log P(c \mid x_{1:k}) \right].$$

Because the grammar is fully enumerable, we evaluate H_k exactly by grouping all 1024 trees by prefix and weighting each next-token distribution by its prefix mass $P(x_{1:k})$ (`conditional_entropy_floor`). Only the $d - 1 = 7$ leaves that have a non-empty left context are predicted (the first leaf carries no information and is not scored), and the reported floor is their mean,

$$\bar{H} = \frac{1}{d-1} \sum_{k=1}^{d-1} H_k.$$

The **uniform baseline** is the loss of the context-free predictor $q(c) = 1/v$, whose cross-entropy against any next-token law is $-\sum_c P(c) \log q(c) = \log v$ at every position and hence in the mean; knowing nothing costs $\log 8$ nats.

The Bayes-optimal floor. The value of an enumerable grammar is that we know the best achievable loss. Averaging the ground-truth per-position posterior entropy over the seven predicted positions gives the Bayes-optimal mean next-token cross-entropy: 0.725 nats. The uniform baseline is $\ln 8 = 2.079$.

Ground truth posterior. Given a leaf prefix of length k , the posterior over any hidden latent (including the root class) is computed by sum-product belief propagation on the tree: upward messages from observed leaves combine through the production rules to give the marginal over each latent. The recorded self-test (a representative string) shows the root posterior sharpening with context (entropy 1.89 nats at $k = 1$ falling to 0.00 (a single consistent root) by $k = 3$) which previews the blooming geometry below.

2.2. Model and training

We train a HuggingFace `GPT2LMHeadModel` with 2 layers, `n_embd` = 128, 4 heads, and all dropout disabled, totalling 398,848 parameters. There is no tokenizer: RHM leaf symbols are integers fed directly as `input_ids`. This `vocab_size` = 8 and `n_positions` = 8. We train by next-token prediction on 922 training strings (102 held out) for 4000 steps on Apple MPS.⁴

The trained model reaches a final held-out cross-entropy of **0.880 nats** (Figure 1), closing 89% of the gap between the uniform baseline and the Bayes-optimal floor. The model has approached the ground truth posterior predictor. Does it represent the posterior internally?

using nats
so often feels
weird, but,
well.

anyone
needs maths
here? Not
sure worth
adding:
written
everywhere
these days.
Maybe,
say, "for-
mally it is
this" (also,
who knows
maybe HF
silently
added some
fancy tricks
there?)

⁴In Appendix 1 more model configurations are explored (to no qualitative change).

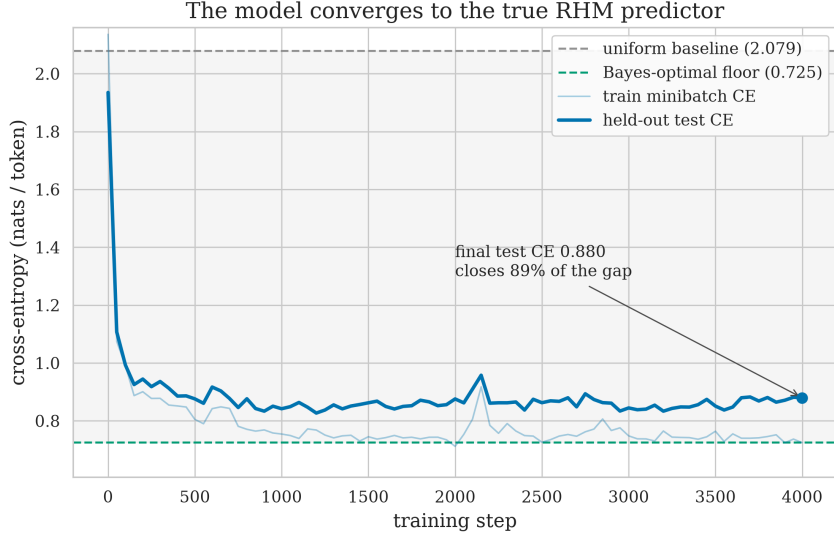


Figure 1: The transformer converges close to the ground truth RHM predictor. Held-out next-token cross-entropy (heavy line) falls from the uniform baseline ($\ln 8 = 2.079$, top dashed) toward the Bayes-optimal floor (0.725, bottom dashed) over 4000 training steps, ending at 0.880 nats, closing $\approx 89\%$ of the uniform $\rightarrow$ Bayes gap (shaded). The faint line is the train-minibatch CE.⁵

2.3. Pre-registered prediction

Following the project’s honor-code rule, we committed our predictions to git **before** training any model or computing any probe (PREREGISTRATION.md). In brief, we anticipated: (1) the root posterior is **linearly decodable** from the residual stream, with $R^2 > 0.5$ at the final layer and a shuffled baseline near 0; (2) the belief **blooms**: early positions cluster near the prior, late positions spread toward simplex vertices, with posterior entropy falling and activation radius growing with context k ; (3) decodability **accumulates additively across depth**, lowest at the embedding and highest after the last layer; and (given aforementioned expectations are optimistic wrt. exact belief being recoverable from residual stream) (4) the representation is **causally used**, so steering the residual along a belief direction shifts the output toward the corresponding latent’s leaves (that is, parse tree reflects the steering: empirically the steered-towards-vertex being actually part of the parse tree). We also registered alternative outcomes (degenerate collapse, non-linear-only encoding, MAP-only encoding, flat-with-depth) as falsification handles. We report against these predictions in Section 3.6.

what belief, exact or model’s recovered?

radius grows bc activations go tow. simplex vertices?

3. Results

3.1. A single linear probe partially decodes the root posterior

We fit one global least-squares affine map from the 128-d residual stream to the 8-class ground truth root posterior space, on a probe set of $N = 400$ held-out examples, and score it by R^2 on held-out data. A single probe is fit once on all context positions pooled; we then score that same fixed probe both pooled and, separately, at each context position. Because the train/test split is over whole sequences, every held-out sequence contributes one example at each of the eight positions, so each per-position score still rests on all $N = 400$ examples. The final-layer probe reaches $R^2 = 0.38$, while at the control task (the same probe fit to **shuffled** labels, following Hewitt & Liang (2019)) scores ≈ 0 (-0.085). So the ground truth posterior is linearly accessible above chance, but the probe explains only a third of the variance: the recovery is **partial**. ($R^2 = 0.38$ means the affine map accounts for 38% of the variance in the ground truth posterior vectors, where 1 would be exact reconstruction and 0 no better than always predicting the mean posterior.) Projected into a belief-space PCA basis (Figure 2), the probe’s predicted posteriors **visibly** occupy the same structured region as the ground truth posteriors and trace the same position gradient — a qualitative correspondence read off the projection, not a computed alignment score — but as a diffuse

⁵Seeded reproduction of the canonical run, results/refrun/; exact parameters in Appendix 9 (Stage 2).

cloud rather than the discrete point set of the ground truth.⁶ The affine correspondence is particularly weak for the root; it strengthens markedly for the deeper latents — mid-level R^2 of 0.51 and 0.59, deepest up to 0.66 (Figure 4, next subsection).

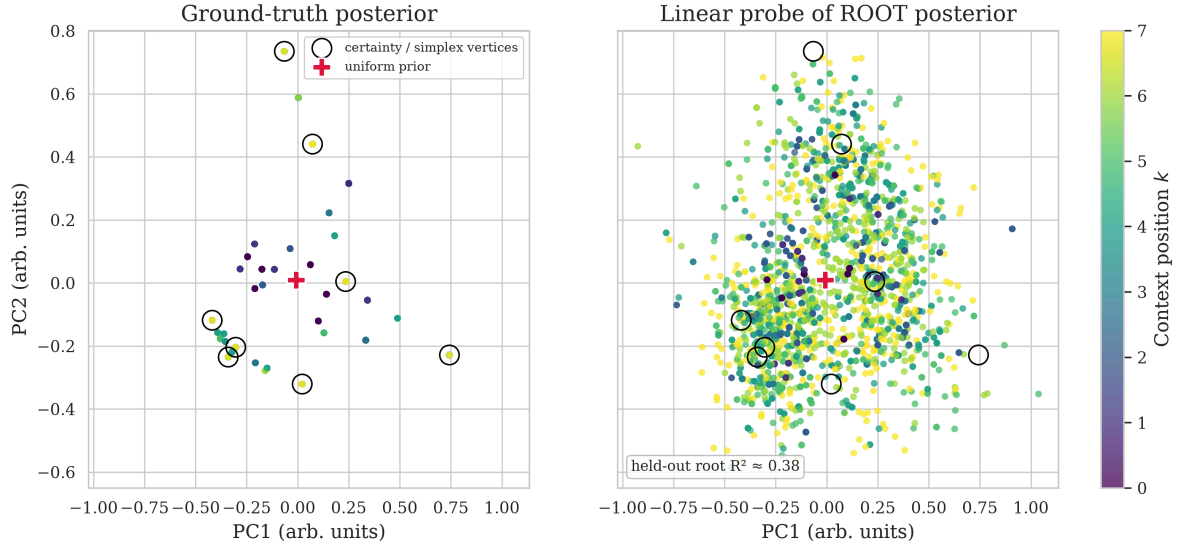


Figure 2: The residual stream is a **partial** affine image of the exact belief simplex. **Left:** PCA(2) of the ground truth root posteriors (few distinct values, hence sharp overplotted dots); small- k points sit near the prior, large- k points spread toward vertices. **Right:** the linear probe’s predictions **for the root posterior** in the same basis, colored by context position — the same region and a similar position gradient, but a diffuse cloud: held-out root $R^2 \approx 0.38$, an imperfect recovery, not an exact one. Both panels are the **root** class specifically.

3.2. Root posterior decodability accumulates across model depth and context ix

The residual stream serves as a running sum of layer contributions, so we ask where the belief is built. Both panels here decode the **root** posterior. The residual stream is read at three points. A GPT-2 “block” is one transformer layer — self-attention (here 4 heads) followed by an MLP, both writing into the residual stream — and our model stacks two such blocks. Requesting `output_hidden_states` returns $n_{\text{layer}} + 1 = 3$ residual snapshots: index 0 is the token+position embedding (the input to block 1), index 1 is the residual after block 1, index 2 after block 2. The four heads live **inside** each block and are not separate readout points; the “Layer 0/1/2” axis is these three **belief-readout points** (the sites where we decode the posterior), not three transformer layers. Root-posterior probe R^2 rises along them: -0.00 at the embedding, 0.15 after block 1, 0.38 after block 2, while the shuffled control stays at ≈ 0 throughout (Figure 3, left) — each block adds belief-relevant signal. Resolving the same root probe by context position (Figure 3, right), the root is already decodable at early positions even at the shallow readout points, and at the final readout it is near-perfectly decodable ($R^2 = 1.0$) for the first few positions 0–3.⁷

I have a feeling this is beautiful and should have been predicted, but I didn't predict it initially. It would be nice to derive this. Also, would adding another layer help? We should clearly link these results.

⁶The ground-truth panel looks sharper partly because the ground truth posterior takes few distinct values, and identical points overplot, whereas every probe prediction differs slightly.

⁷A few mid-readout cells are strongly negative (the probe underperforms the mean predictor on those positions); the heatmap colors are clipped for legibility, but the cell labels show the raw values

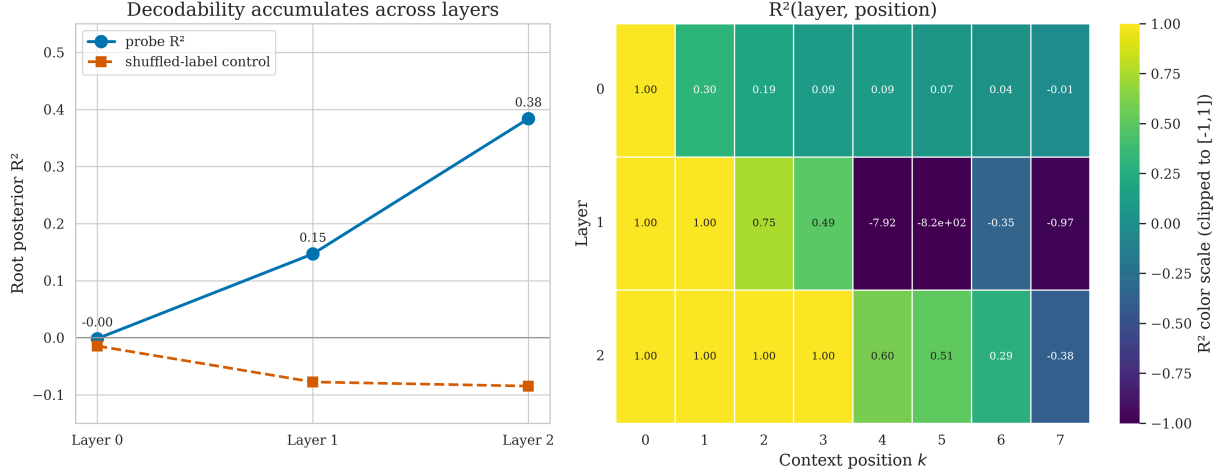


Figure 3: Root-posterior decodability accumulates across residual depth and context. Both panels decode the **root** class; “Layer 0/1/2” are the three residual readout points (embedding, after block 1, after block 2 = $n_{\text{layer}\{+ \}}1$). **Left:** root-posterior R^2 rises along them ($-0.00 \rightarrow 0.15 \rightarrow 0.38$) while a shuffled control stays at ≈ 0 . **Right:** R^2 (readout point, position) heatmap; the final readout reaches $R^2 = 1.0$ on the earliest positions.

3.3. Non-root latents are more strongly decodable than the root

The root is only one of the tree’s hidden latents. Probing for the ground truth posterior over latents at **every** tree level resembles a gradient with some node-level variation (Figure 4). Precisely, root (L_0) $R^2 = 0.38$; the two level-1 mid latents 0.51 and 0.59; and the four deepest level-2 latents 0.61, 0.66, 0.50, and 0.66. The residual encodes the entire hierarchy, but the local, near-leaf latents, which are more directly predictive of the next token, are usually read off more cleanly than the coarse global root. Root is the **hardest** latent to decode, likely because it is the most abstract thus least locally predictive.

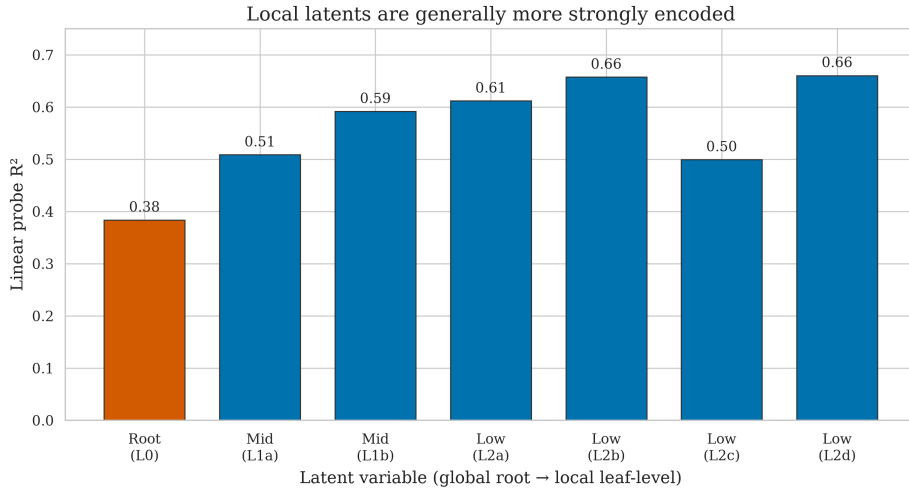


Figure 4: The residual encodes the whole latent hierarchy, deeper/local latents most strongly. Probe R^2 is lowest for the root ($L_0 = 0.38$), higher for level-1 (0.51, 0.59), and generally higher for level-2 (0.61, 0.66, 0.50, 0.66).

However (An important refinement based on robustness studies (from Appendix 1), this image is refined during robustness analysis. Scaling the identical per-level probe to $L = 4$ (the depth study in Figure 17) shows the gradient is really an **inverted U**: the mid-level latents decode strongest, while both the coarse root and the most-local leaf-parents fall off.

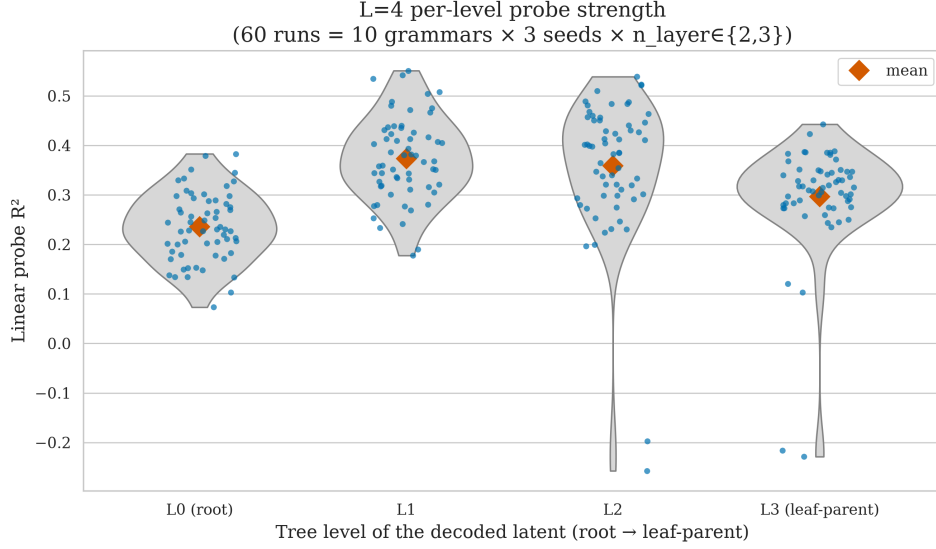


Figure 5: Preview of the inverted-U refinement (full detail in Appendix 1.3). At $L = 4$, per-level probe R^2 across 60 runs (random seed and the exact grammar used vary) rises from the root (L_0) to the mid latents (L_1, L_2) and falls back toward the leaf-parents (L_3) — an inverted U, not a monotone “deeper is stronger” ladder.

3.4. The belief readout blooms with context

Our central geometric result (Figure 6): projecting the linear belief readout into a belief-space PCA(2) basis, the points form a tight central cluster near the uniform prior when little context is observed and expand outward toward the simplex vertices as context accumulates. The mean radius about the prior anchor grows from 0.17 at $k = 0$ to ≈ 0.39 at $k = 7$, tracking the true posterior’s own growth ($0.17 \rightarrow 0.47$); equivalently, the ground truth posterior entropy falls with position (1.84 nats at $k = 0$ to 0.00 at $k = 7$). The right panel colors the same cloud by ground-truth root class: it is only **loosely** organized by class — consistent with the modest root decodability ($R^2 = 0.38$, MAP root accuracy 0.47, Figure 21) — not a clean separation. Raw residual PCA, dominated by token and position nuisance variance, does **not** bloom — that is, its distance from the center does not grow with context position k (radius-vs- k correlation ≈ 0.03 , against ≈ 0.81 for the belief readout) — it is the **belief content** of the residual that does.

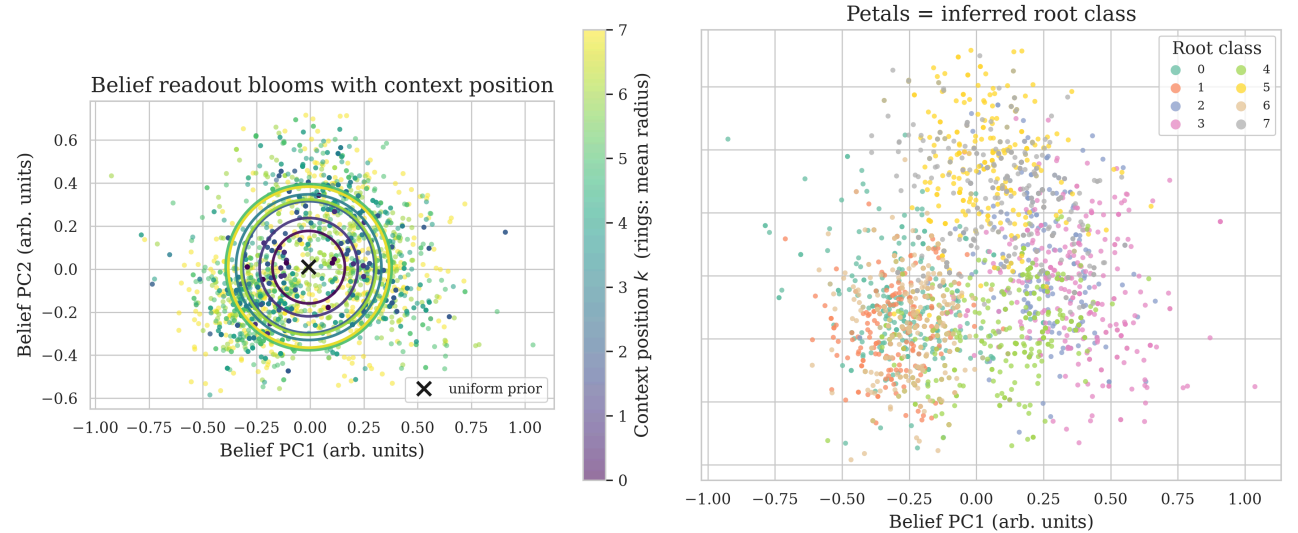


Figure 6: The belief read out of the final-layer residual blooms with context. **Left:** points colored by context position k with rings marking each position’s mean radius about the prior ($\times$); the cloud expands outward as context grows (mean radius $0.17 \rightarrow 0.39$, tracking the true posterior $0.17 \rightarrow 0.47$). **Right:** the same cloud colored by ground-truth root class is loosely organized by class (partial, not clean separation — root MAP accuracy 0.47).

3.5. Causal steering: the belief is used, not just decodable

A representation can be decodable yet causally inert. To test for this we apply a patch to the layer-1 residual, pushing it by α times the belief direction toward a chosen latent of our choice, and measure

the effect on the next-token distribution (Figure 7). Steering **toward the true** latent leaves predictions untouched (the model already holds that belief): the true token’s log-probability stays at -0.73 for all α . Steering **toward some wrong** latent collapses the mean log-probability of the true next token from -0.73 to -8.5 as steering effort α grows, and re-routes probability mass onto the wrong latent’s children: the next-token probability the model assigns to exactly the leaf symbols that the steered-to (wrong) latent can emit at that position (summed softmax mass over those symbols, averaged across positions and examples) rises from 0.22 to 0.53.

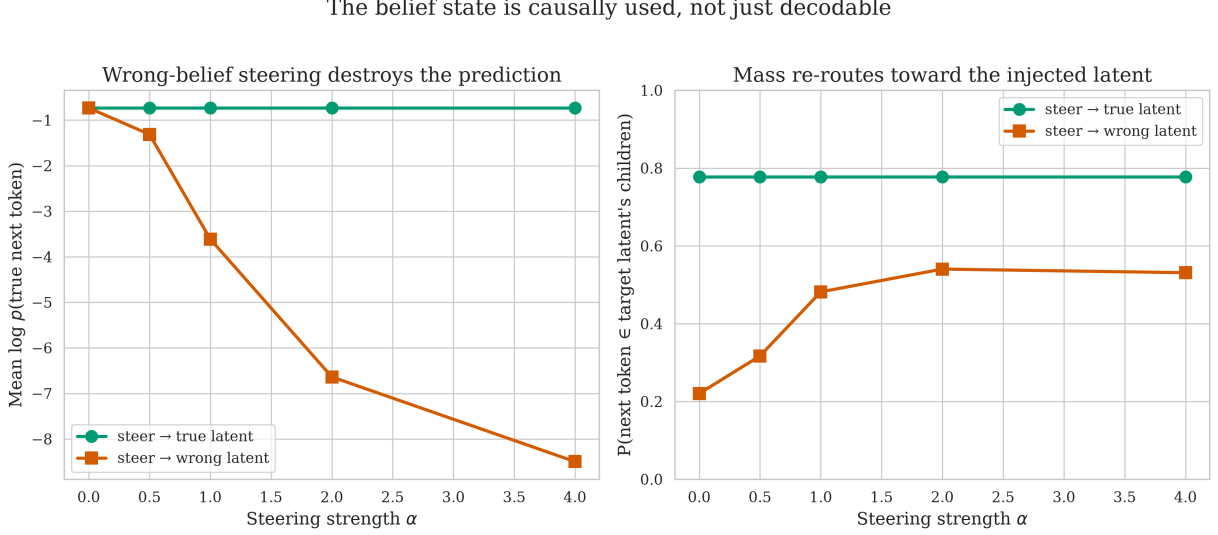


Figure 7: The belief state is causally used. Steering the layer-1 residual toward the **true** latent leaves predictions intact (green); steering toward a **wrong** latent collapses the true next token’s log-probability (left, orange: $-0.73 \rightarrow -8.5$) and re-routes mass onto the wrong latent’s children (right, orange: $0.22 \rightarrow 0.53$).

3.6. Pre-registration scorecard

We grade each pre-registered prediction honestly against the outcome (Table 2).

Prediction	Outcome	Verdict
Linear decodability, root $R^2 > 0.5$, shuffled ≈ 0	Root $R^2 = 0.38$ (shuffled ≈ 0). Decodable and well above baseline, but below the 0.5 threshold for the root; mid/deep latents do exceed 0.5 (up to 0.66).	Partial
Blooming with context (radius $\uparrow$, entropy $\downarrow$)	Mean radius $0.17 \rightarrow 0.39$; posterior entropy $1.84 \rightarrow 0.00$.	Hit
Monotone layer-wise accumulation of R^2	$-0.00 \rightarrow 0.15 \rightarrow 0.38$ across the three residual indices.	Hit
Causal usage via steering (ordered effect)	Steering $\rightarrow$ wrong: log p collapses $-0.73 \rightarrow -8.5$, monotone in α ; target-child mass $0.22 \rightarrow 0.53$.	Hit

Table 2: Scored pre-registered predictions (PREREGISTRATION.md, committed before any result). Three of four held; the lone miss is the root-specific $R^2 > 0.5$ magnitude.

Three of four predictions held cleanly. The single miss is the root R^2 , which came in at 0.38 rather than the anticipated > 0.5 . We take this as a genuine and informative negative: the anticipation was correct in **form** (linear, above baseline, growing with depth and context) but our magnitude estimate for the **root specifically** was optimistic. The latent-level sweep — which we did not pre-specify — explains why: the root is the coarsest, least locally-predictive latent, and the threshold we anticipated is in fact met by every deeper latent in the hierarchy. Three of the four pre-registered failure modes are contradicted by direct measurement: **degenerate collapse** (the readout blooms, and under ambiguity the belief stays non-trivial, Figure 8), **MAP-only encoding** (MAP root accuracy is only 0.47 while the full-simplex

I was over-using monotonicity in my initial guess, which is a very strong claim, to be revised (shouldn't have written it in the first place)

R^2 is positive and graded — a vertex-collapsed representation would give near-perfect MAP accuracy), and **flat-with-depth** (probe R^2 rises $-0.00 \rightarrow 0.15 \rightarrow 0.38$ across readout points). The fourth, a **non-linear-only encoding**, we did not test separately — we fit no nonlinear probe — but the graded **linear** R^2 already establishes a substantial linearly-decodable component, which is what that mode would have denied.

3.7. Brief summary of the robustness study results.

To test whether the observations hold across different setups, we have varied the following:

1. We have run the analysis against more grammars satisfying our initial conditions.
2. We have performed the same analysis for different model setups, varying the number of attention heads and layers, as well as the hidden dimension size and training duration.
3. We have considered deeper grammars of layer 4.

Main observations of this section held. Respective setup-specific and sweep-specific observations can be seen in the Appendix 1.

4. Engineering non-trivial belief geometry

Everything so far lives on a tree whose belief, given the full leaf string, collapses to **certainty**: with uniform unambiguous rules the eight leaves invert level-by-level to exactly one root, so the $k = 8$ posterior is a delta and the belief path runs from the prior straight to a simplex vertex. Yet Shai et al. (2026) considered another setup: there, for HMMs, belief never collapses to a single certain state. There, belief traces a fractal (Sierpinski-like, self-similar, recurrent) attractor. But RHM has no recurrence, so “slow forgetting” — an HMM’s belief only gradually losing information about the distant past — has no analog. A possible analog of “the state can’t be restored” is a **non-invertible observation channel**: a grammar where even the full leaf string leaves the root uncertain, so the reachable-belief set is a non-trivial attractor in the simplex rather than a path to a vertex.

We add two per-level knobs to the grammar (`rhbm.py`, both reducing exactly to the canonical RHM at their off setting).

- **Ambiguity** ρ shares a fraction of the $v \cdot m$ rule entries’ child-tuples **across parents** (many-to-one leaf $\rightarrow$ root structure); $\rho \in \{0, 0.3, 0.6\}$. This knob actually introduces non-invertibility of the latents.
- **Skew** draws each parent’s rule-choice probabilities non-uniform from a Dirichlet with concentration α : **none** is the canonical uniform $1/m$; **mid** uses $\alpha = 1.0$ and **high** uses $\alpha = 0.2$ (smaller $\alpha \Rightarrow$ more peaked, near-deterministic rule choice). This is a control node that just complicates the invertibility logic with uneven branching.

Both belief propagation and the brute-force reference (both derived in Section 2.1) were generalized to **weighted** sum-product using the same probabilities the sampler uses; the BP $\leftrightarrow$ brute-force agreement holds to 2.5×10^{-16} under both knobs simultaneously. We sweep the full 3×3 grid $\times 3$ rule-table draws — **27 runs**⁸.

We naturally observe the mean ground truth $k = 8$ root-posterior entropy is 0 at unambiguous setup regardless of the skew. It rises monotonically with ambiguity ρ growth: $0.00 \rightarrow 0.71 \rightarrow 1.38$ nats at skew=none (Figure 8). At $\rho = 0$ the belief collapses cleanly to 0 by $k = 8$ (the bloom-to-vertex), while at $\rho = 0.3$ and $\rho = 0.6$ it reaches positive floor.

⁸at the pinned published arch (`run_noncollapse.py`, writing `results/noncollapse/`), with the (none, none) cell reproducing the pass-1 baseline

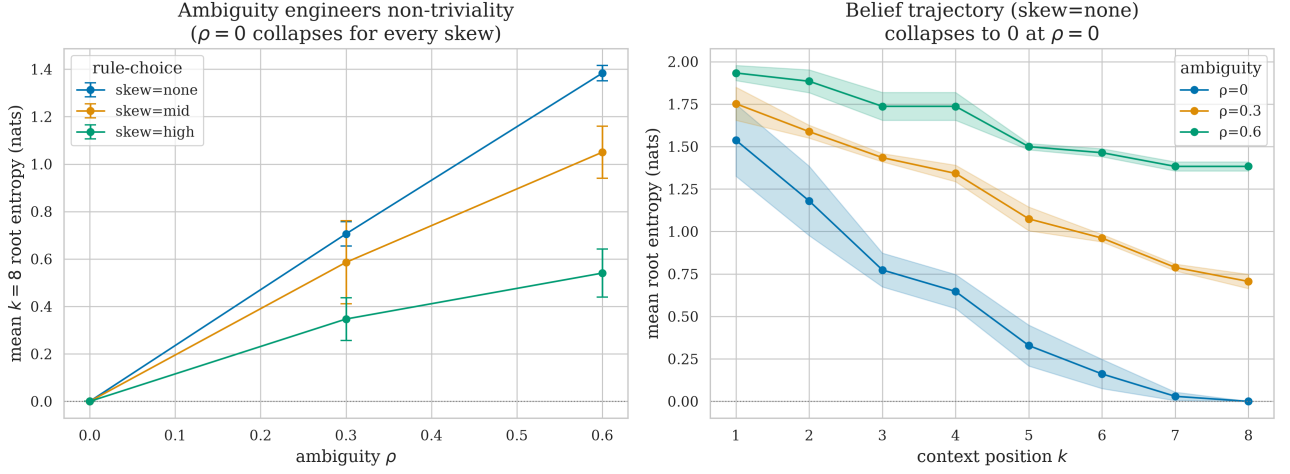
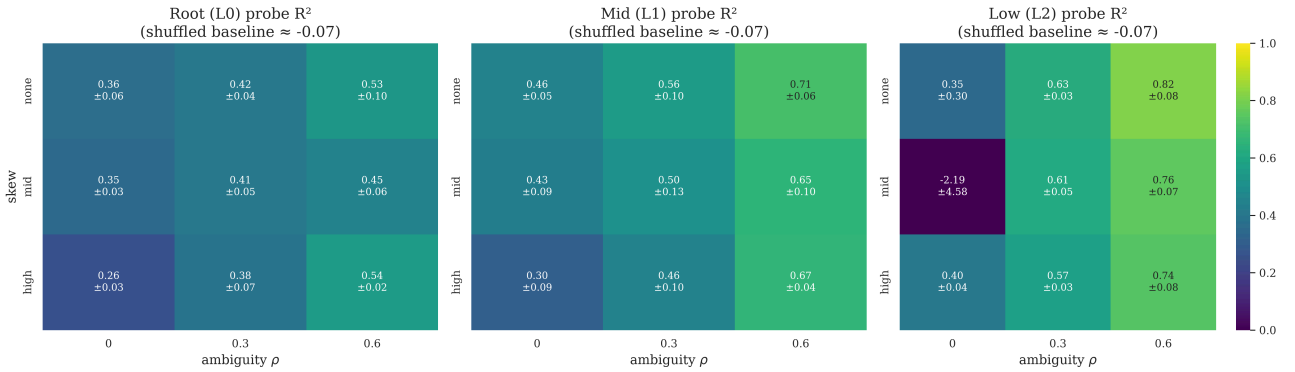


Figure 8: Ambiguity, not skew, engineers non-triviality. **Left:** mean ground truth $k = 8$ root posterior entropy vs ambiguity ρ , one line per skew (error bars over 3 draws). At $\rho = 0$ entropy is exactly 0 for **every** skew — skew alone does not prevent collapse, and it rises monotonically with ρ . **Right:** the full entropy-vs-context trajectory at skew=none; at $\rho = 0$ the belief collapses to 0 by $k = 8$, at $\rho > 0$ it reaches a positive floor.

The linear probe sharpens with uncertainty growth! At every cell of the grid the residual stream still linearly encodes the (now spread) posterior well above the shuffled-label control task of ≈ -0.07 (Figure 9). That control task is the same probe refit to randomly **permuted** labels and scored on held-out data, averaged over the 27 runs. The probe did not **degrade** as ρ rose: R^2 **rises** with ambiguity, $0.36 \rightarrow 0.42 \rightarrow 0.53$ at skew=none, and the mid- and low-level probes rise even more steeply (L_1 : $0.46 \rightarrow 0.71$; L_2 : $0.35 \rightarrow 0.82$).



this leaves an interesting question. Why is that? Does this indeed hint against memorization?

Figure 9: The linear probe survives ambiguity and skew, and **sharpens** with ambiguity. Per-level probe R^2 (root L_0 , mid L_1 , low L_2) over the 3×3 skew $\times$ ambiguity grid; each cell is mean $\pm$ SD over 3 rule-table draws, against a shuffled baseline of ≈ -0.07 . R^2 rises left-to-right (with ρ) at every level. The single dark L_2 cell at (mid, $\rho=0$) is a near-deterministic-target instability, not a decodability failure.

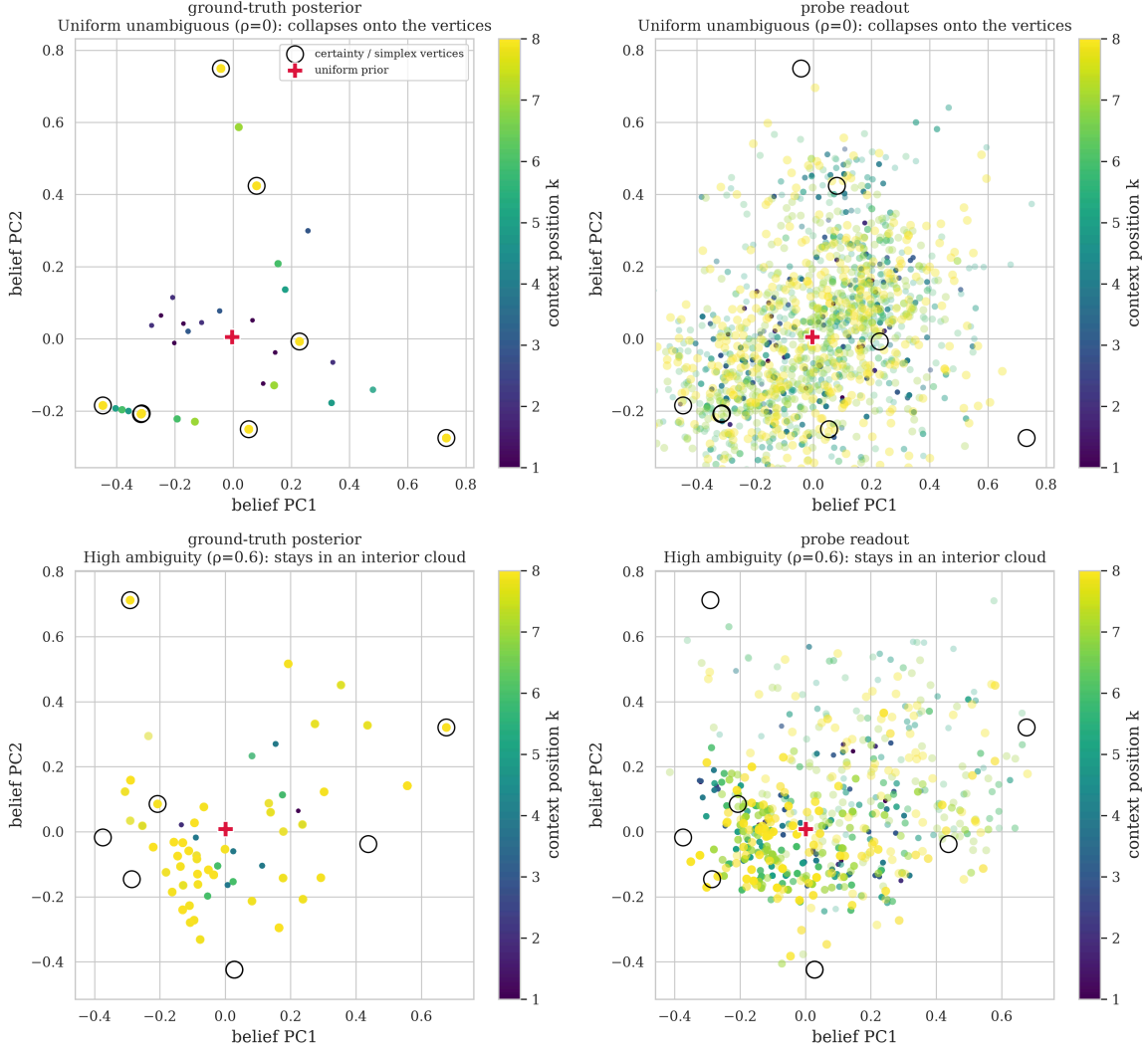


Figure 10: Ground truth posterior and linear-probe readout after PCA, colored by context position k for the uniform corner $\rho = 0$ and the high-ambiguity corner $\rho = 0.6$. At $\rho = 0$ the late-context exact beliefs land **on** the vertices (due to collapse to certainty), which is not always the case at $\rho = 0.6$. Probe readouts occupy roughly same regions as their counterpart posteriors, with uncertainty case readouts being more aligned with the ground truth posterior. (For vizualization purposes, shown data is from one single run).

5. Discussion and limitations

Our key findings are thus the following: **(i)** Decoder-transformer language model trained to near-Bayes-optimal loss on a hierarchical grammar carries a **partial** linear image of the exact belief over the grammar’s hidden latents in its residual stream. This image is most fuzzy for the grammar root (most global latent of the grammar) ($R^2 = 0.38$), sharper for the other, especially mid-level latents (R^2 up to 0.66). **(ii)** That partial linear image gets assembled additively across layers, with belief readout blooming from the prior toward the vertices as context grows (evidence accumulates). **(iii)** Belief readout is causally relied upon at generation time. **(iv)** The belief recovery is impartial despite the exact belief state being known. **(v)** Uncertain grammars (the ones where the exact belief state is not a delta) are **better** decodable than the unambiguous ones, and probe sharpens as uncertainty grows.

This reproduces the core Simplex belief-geometry phenomenology — linear decodability, simplex geometry, and causal use — in a setting where the exact belief state is known. The match is at the level of phenomenology, not of fidelity or mechanism: our linear recovery is **partial** ($R^2 = 0.38$ for the root), and Shai et al. (2026) likewise report a probed, imperfect image rather than perfect recovery. The generative settings differ too — their HMM mixes toward a fractal attractor, whereas the RHM tree collapses deterministically to a simplex vertex at full context.

5.1. Speculations

Oleg:

- what if we give some padding post-string. Will the results differ?
- The root is worst decodable. This is due to attention having to re-store information about the tree structure, and for some reason it did not happen within one layer. Kinda beautiful, unclear why.
 - AI says:
This is intuitive in hindsight — next-token prediction rewards representing whatever is most locally predictive — but it sharpens the belief-geometry claim: the residual stream tracks the full latent posterior, weighted toward what the task needs, not a single privileged variable.

5.2. Desired next steps

- What if we consider smaller alphabets?
- Normal language is more similar to HMMs because it allows for recursion. I think the belief propagation would be more complicated.
- The role of enumerability: does the model learn a **general** belief-geometry principle, or does it just memorize the exact belief for this grammar?

Limitations. (i) The grammar is deliberately tiny and fully enumerable ($s = 2$, $L = 3$, 1024 trees); this is what makes the beliefs exact and the verification airtight, but it leaves open how the geometry scales to larger, non-enumerable RHM. (iii) The strongly-negative mid-layer probe cells indicate the affine probe is locally mis-specified at some position/layer combinations; a per-position or whitened probe would tighten these estimates. (iv) Steering is applied at a single layer (layer 1) along a mean-difference axis; a learned causal direction and a layer sweep would strengthen the causal claim. (v) The detailed pass-1 figures (Figure 2–Figure 7) are from a single training run and grammar draw; the grammar-sweep and architecture-sweep sections quantify how the **headline** metrics move across 10 grammars / 3 seeds and across 11 capacity configs / 3 grammars / 3 seeds respectively, but the architecture sweep is one-axis-at-a-time (no $n_{\text{layer}} \times n_{\text{embd}} \times n_{\text{head}}$ interaction cells). The $L = 4$ section extends the family one tree level deeper; still-larger grammars ($L \geq 5$, larger v , $m > 2$), where brute-force belief verification becomes intractable, remain a future pass.

6. Conclusion

On an exactly-solvable hierarchical grammar, a small transformer’s residual stream is a linear image of the exact Bayesian belief simplex: it accumulates across depth, blooms with context, encodes the whole latent hierarchy (local latents most strongly), and is causally used.

strictly

7. References

- Cagnetta, F., Favero, A., Sclocchi, A., & Wyart, M. (2025). *Scaling Laws and Representation Learning in Simple Hierarchical Languages: Transformers vs. Convolutional Architectures*. <https://arxiv.org/abs/2505.07070>
- Hewitt, J., & Liang, P. (2019). *Designing and Interpreting Probes with Control Tasks*. <https://arxiv.org/abs/1909.03368>
- Shai, A., Amdahl-Culleton, L., Christensen, C. L., Bigelow, H. R., Rosas, F. E., Boyd, A. B., Alt, E. A., Ray, K. J., & Riechers, P. M. (2026). *Transformers learn factored representations*. <https://arxiv.org/abs/2602.02385>

Project sources: PREREGISTRATION.md, EXECUTION_OUTPUT.md, results/analysis.json, results/root_reconstruction.json, artifacts/train_summary.json. Grammar sweep: sweep.py, sweep.yaml, results/sweep/g*/{config.json, analysis.json}, figures/sweep_sanity.csv. Architecture sweep: run_arch.py, results/arch/g*/{config.json, analysis.json}, figures/arch_sanity.csv.

1. Appendix: Robustness studies

The main text rests on one grammar and one training run, so this appendix asks whether the findings are properties of the RHM family rather than of a single draw or a single architecture. It reports three sweeps: across grammars (10 grammars $\times$ 3 seeds), across model capacity (11 architecture configs $\times$ 3

grammars $\times$ 3 seeds, one axis at a time), and one tree level deeper ($L = 4$; 10 grammars $\times$ 2 model depths $\times$ 3 seeds). The four core findings — the inverted level gradient, blooming, layer accumulation, and causal steering — replicate across all ten grammars with low replicate variance, and the decodability holds across capacity (rising with width and depth, weakest dependence on head count). The depth-4 sweep further refines the level gradient into an **inverted U**: mid-level latents decode most strongly, with both the coarse root and the most-local leaf-parents falling off.

1.1. Generalization across grammars

The results above come from a single grammar⁹ and a single training run. To test whether they are properties of the RHM **family** under the same fixed constraints¹⁰ rather than artifacts of one rule draw, we sweep the content of the rule table¹¹ across ten independently sampled grammars with three replicate training seeds each, resulting in 30 runs. We have ensured that the sampled grammars aren't isomorphic by post-factum ensuring that the respective nodes adjacency tables are not equivalent up to a per-level symbol permutation.¹² The model architecture / training code remained the same as in previous experiments.¹³

- **All grammars train to near-Bayes.**

Every run closes 0.82–0.93% of the uniform to Bayes-optimal loss gap (mean 0.89 ± 0.03), in line with the initial observation. The runs details can be seen in Table 5.

- **The root remains the weakest-decoded level** (Figure 11).

Pooling probe R^2 by tree level, the root (L_0) averages 0.35 ± 0.07 (range 0.24–0.48), well below the mid (L_1 , mean 0.50) and leaf-parent (L_2 , mean 0.49) latents. The deepest level shows the widest spread (one grammar dips slightly negative).

- **Blooming and belief-sharpening alongside context hold for every grammar** (Figure 12).

The ground truth posterior entropy falls with context position k in all ten grammars, and the belief readout's radius grows with context.

- **Causal steering replicates with a stable effect size** (Figure 13).

Steering the layer-1 residual toward a wrong latent collapses the true next token's mean log-probability in every run: as the patch strength α increases from 0 to 4, that log-probability drops by 6.9 ± 0.5 nats (mean $\pm$ SD across the 30 runs), with low run-to-run variance.

In short, all four earlier findings – poor root decodability, blooming, layer-wise accumulation (recomputed in every run's per-layer probe), and causal steering – replicate across the grammar family.

investigate:
not a failed
training,
poor train-
test split?

node-level
rule
structure
matters
most for the
most local
latents?

⁹grammar = 0

¹⁰Reminder: fixed constraints so far are ($s = 2$, $L = 3$, $v = 8$, $m = 2$, uniform unambiguous rules)

¹¹See the example (first tried grammar) rule table at Table 1

¹²Test code in `grammar_iso.py`

¹³Reminder: model architecture so far: GPT-2-style transformer, 2 layers of 4 attn heads each, hidden size 128, 4k train steps. A single master seed controls model initialisation, training-data sampling, probe sampling, and the probe split, so replicates measure end-to-end sampling variance. Each run writes a deterministic, self-contained directory (`results/sweep/g{NN}_L2_d128_h4_s{S}/`) holding its grammar, a `config.json` manifest (resolved config, git SHA, timestamp), and per-run metrics — recoverable for this paper independent of any experiment-tracker.

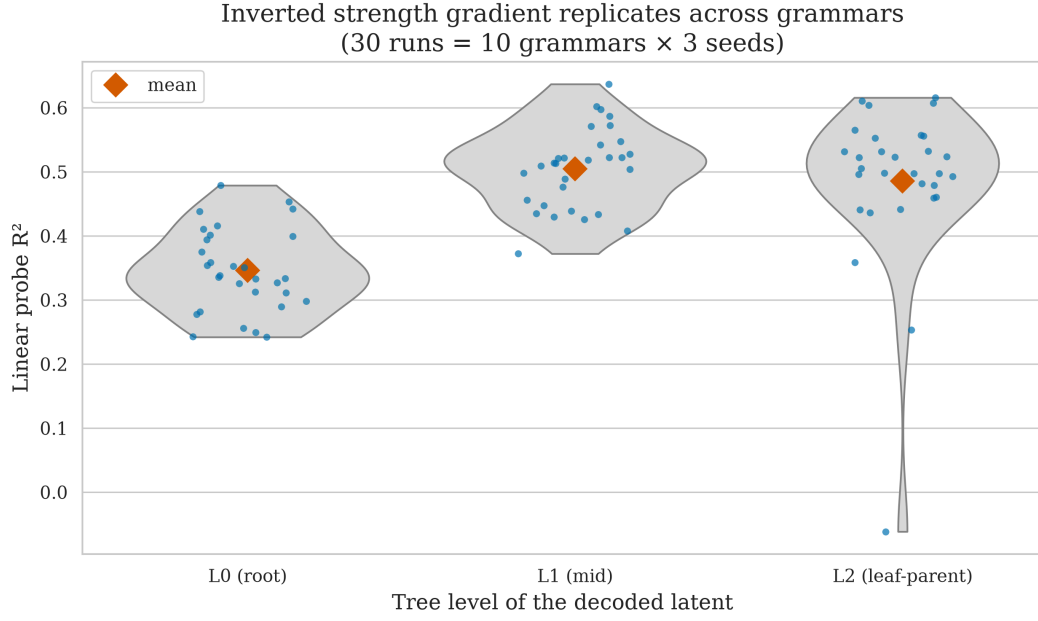


Figure 11: The inverted strength gradient replicates across the grammar family. Each point is one of 30 runs (10 grammars $\times$ 3 seeds); violins show the per-level distribution of probe R^2 , diamonds the means. The root (L_0) is the weakest-decoded level (0.35 ± 0.07), below the mid (L_1) and leaf-parent (L_2) latents (≈ 0.49 – 0.50).

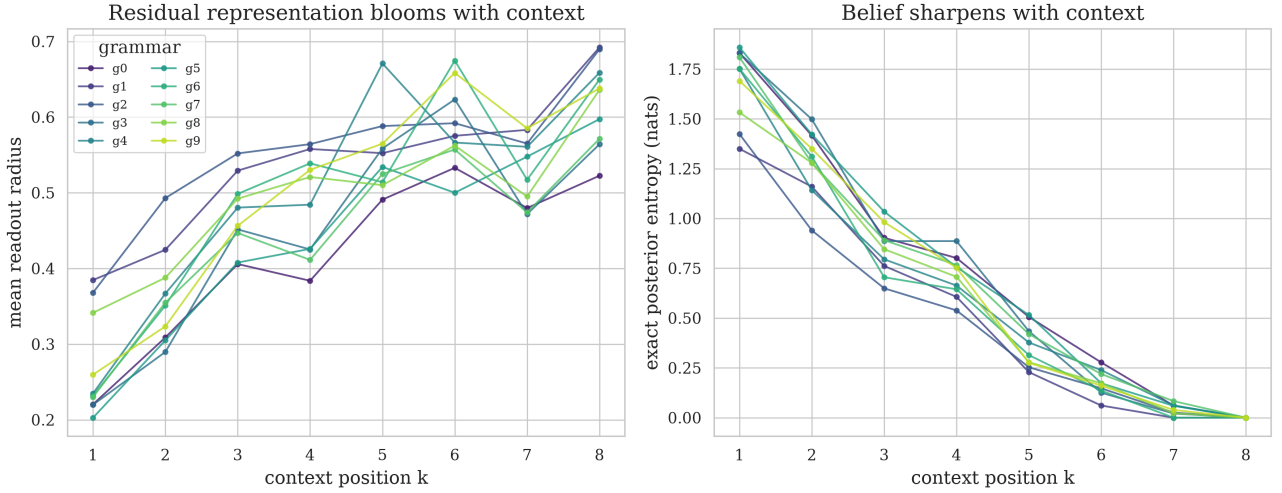


Figure 12: Blooming is family-wide. **Left:** the belief readout’s mean radius grows with context position k (one line per grammar, averaged over seeds). **Right:** the ground truth posterior entropy falls with k for every grammar.

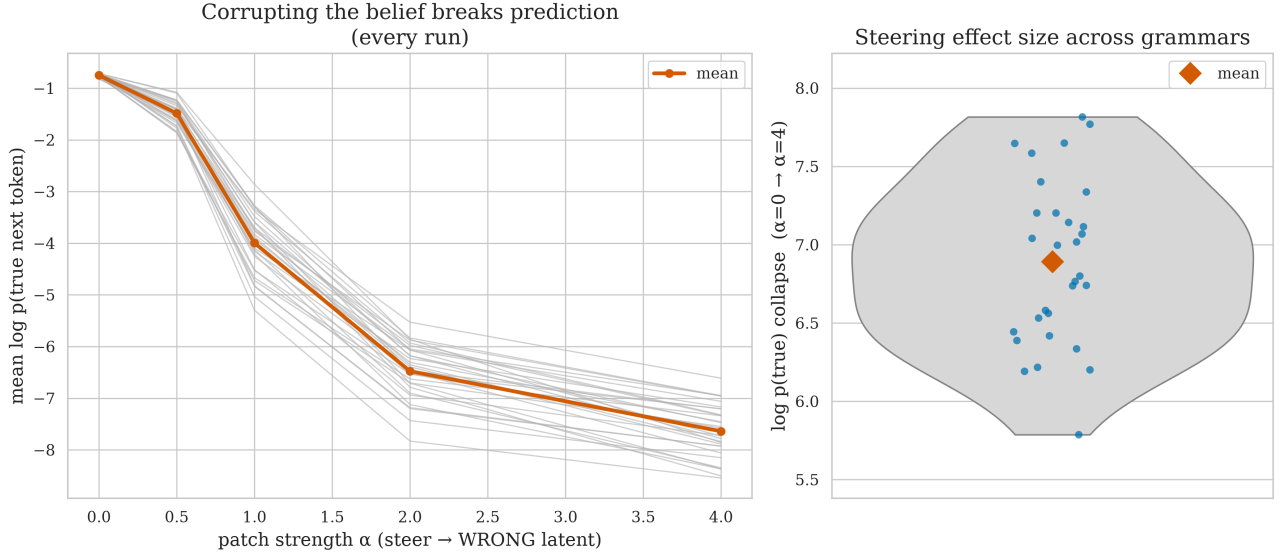


Figure 13: Causal steering replicates with a stable effect size. **Left:** steering the layer-1 residual toward a **wrong** latent collapses the true next token’s log-probability in every run (grey lines), mean in orange. **Right:** the distribution of the collapse magnitude (grey violin, orange diamond = mean) — the drop in the true token’s mean log-probability as $\alpha \{=\} 0 \rightarrow \alpha \{=\} 4$ — across all 30 runs: 6.9 ± 0.5 nats.

1.2. Architecture dependence

The grammar sweep fixes the architecture and varies the data; this section does the opposite. Again, we keep the RHM constraints. We vary the transformer’s capacity one axis at a time around the baseline. We consider (in bold are baseline params) n_{layer} in $\{1, \mathbf{2}, 3, 4\}$, n_{embd} in $\{16, 64, \mathbf{128}, 256\}$, n_{head} in $\{1, 2, 4, 8\}$, and training steps in $\{\mathbf{4000}, 16000\}$. Each axis varies independently with the other three pinned at baseline, the baseline being their common center. Every config is run across $\text{grammar} \in \{0, 1, 2\}$ and $\text{seed} \in \{0, 1, 2\}$ from grammar sweep, **99 runs total**¹⁴. Small models are **expected** to underfit.

Model capacity lifts decodability, with a low-width floor (Figure 14). Width is the strongest lever, across $n_{\text{embd}} = 16 \rightarrow 256$ the root probe R^2 climbs monotonically (from being undecodable at 16): $0.07 \rightarrow 0.21 \rightarrow 0.35 \rightarrow 0.44$, with the mid ($0.12 \rightarrow 0.57$) and leaf-parent ($0.15 \rightarrow 0.59$) levels rising in parallel. Depth lifts every level too: across $n_{\text{layer}} = 1 \rightarrow 4$ the root goes $0.22 \rightarrow 0.42$, the mid $0.32 \rightarrow 0.54$, and the leaf-parent $0.34 \rightarrow 0.48$. Number of heads is the weakest axis: from 1 to 8 heads the root moves only $0.25 \rightarrow 0.36$, the mid $0.37 \rightarrow 0.48$, and the leaf-parent level is essentially flat ($0.42 \rightarrow 0.44$).

Root remains the weakest latent. At all eleven configs the root (L_0) is the weakest-decoded level.

Model depth stretches the build-up and lifts the readout (Figure 15). Plotting root R^2 against normalized residual depth (residual index over n_{layer}), every model rises from ≈ 0.01 at the embedding to its final-layer readout, and deeper models both stretch the accumulation curve and reach a higher endpoint: final-layer root R^2 is $0.22 \rightarrow 0.35 \rightarrow 0.39 \rightarrow 0.42$ for $n_{\text{layer}} = 1$ to 4, respectively.

Longer training has less effect on leaves decodability than on other latents. Quadrupling the training budget ($4000 \rightarrow 16000$ steps) lifts the root by $+0.05$ ($0.35 \rightarrow 0.41$) and the mid level by $+0.05$ ($0.48 \rightarrow 0.53$), but the leaf-parent level by only $+0.02$ ($0.44 \rightarrow 0.47$).

Decodability decouples from loss fit (Figure 16). Across all 99 runs the correlation between `loss_gap_closed` and probe R^2 is modest — 0.46 for the root, 0.42 for the mid, 0.17 for the leaf-parent, and the scatter is wide. A model can reach near-Bayes loss without linearly representing the coarse belief, and vice versa.

interesting

¹⁴`run_arch.py`, writing steps-disambiguated dirs `results/arch/g{NN}_L{n}_d{d}_h{h}_t{steps}_s{S}/`

Marginal capacity effects on belief decodability (OAT around L2·d128·h4·t4000; error bars = SEM over 3 grammars $\times$ 3 seeds)

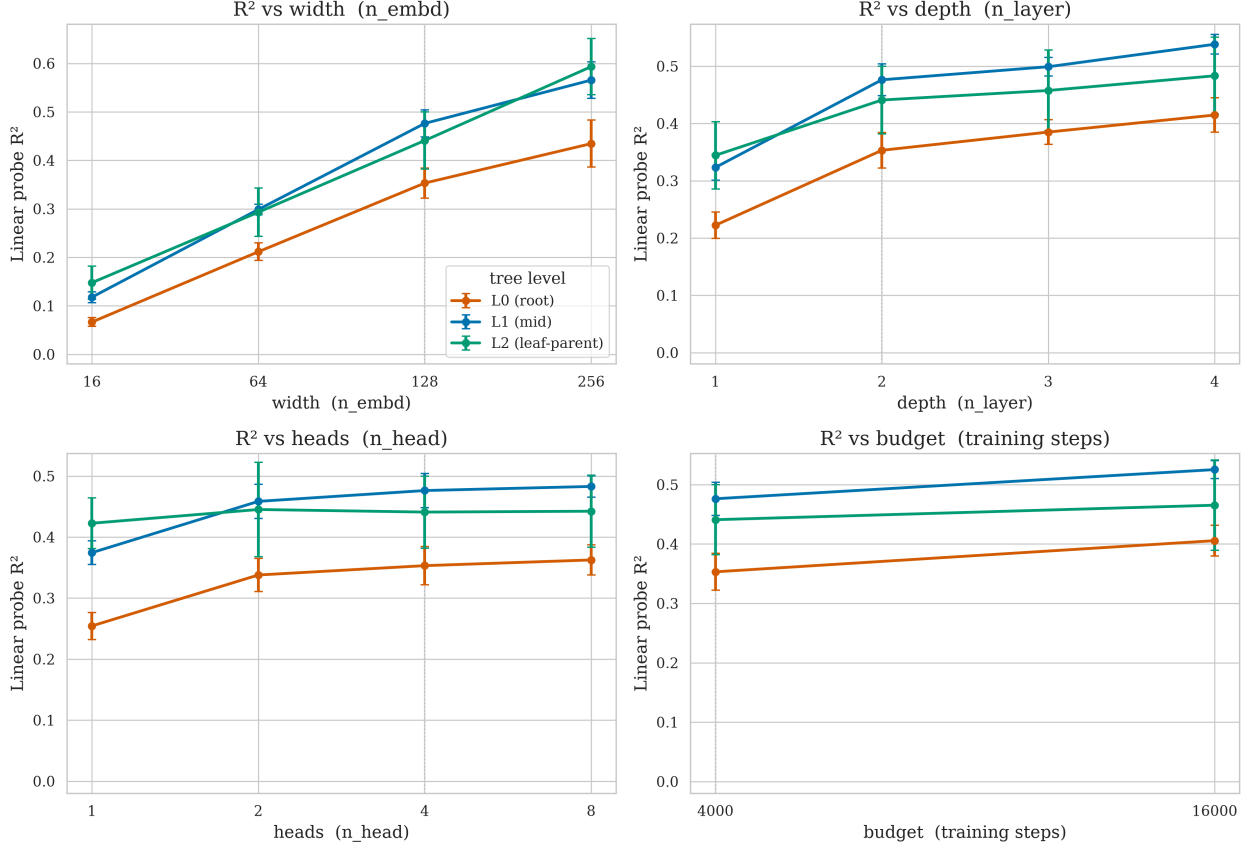


Figure 14: Marginal capacity effects. Each panel varies one axis with the other three at baseline; points are root (L_0), mid (L_1), and leaf-parent (L_2) probe R^2 , error bars are SEM over 3 grammars $\times$ 3 seeds, the dotted line marks the shared baseline. Width and depth lift all levels (root collapses at $n_{\text{embd}} = 16$); heads matter least; the root is the lowest level in every panel.

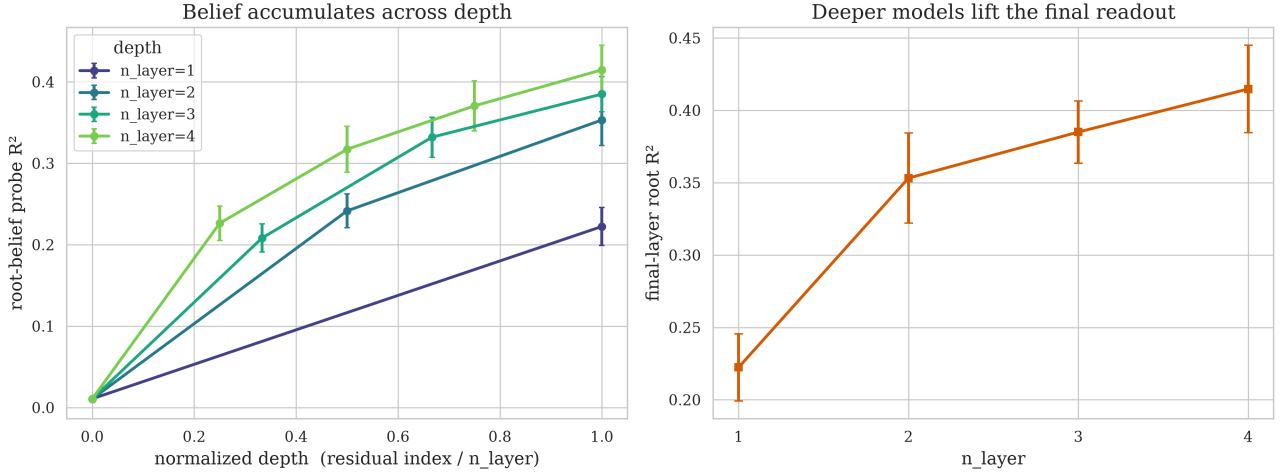


Figure 15: Depth and accumulation. **Left:** root-belief probe R^2 vs normalized residual depth, one line per n_{layer} (mean $\pm$ SEM over grammars $\times$ seeds); every model accumulates from the embedding to its readout, and deeper models reach higher. **Right:** final-layer root R^2 rises monotonically with depth.

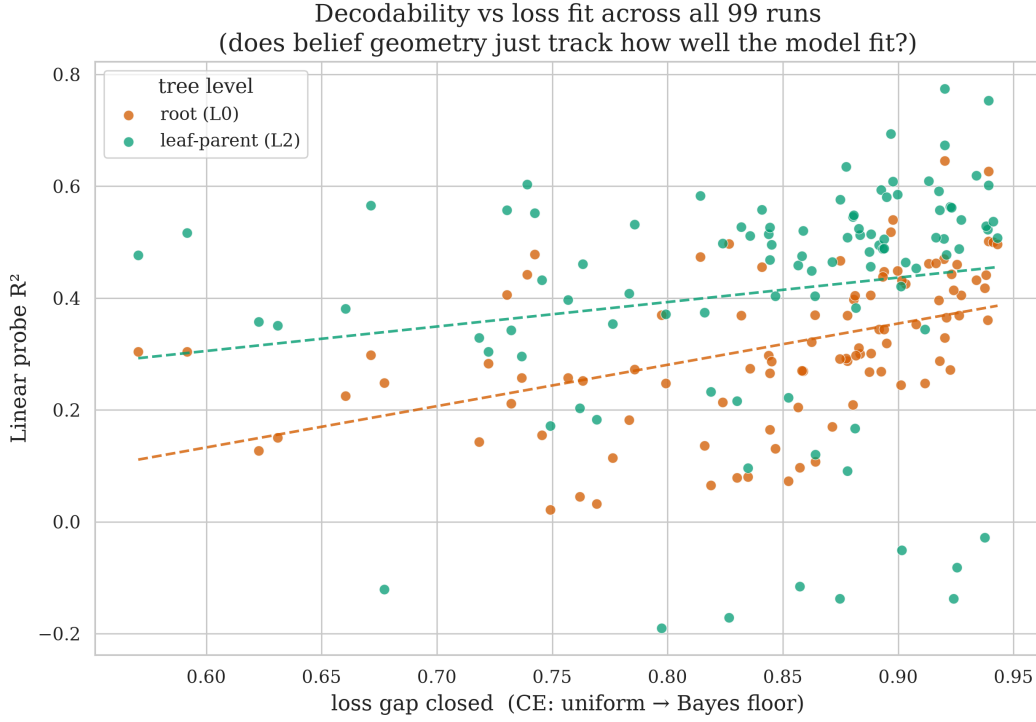


Figure 16: Decodability vs loss fit, all 99 runs. Root (L_0) and leaf-parent (L_2) probe R^2 against `loss_gap_closed` (fraction of the uniform→Bayes CE gap closed). Dashed lines are least-squares fits; the wide vertical spread at fixed loss fit shows decodability is not determined by how well the model fit the loss.

1.3. Going deeper: $L = 4$

The $L = 3$ tree is shallow enough that the root collapses to certainty within a few tokens (Figure 3), compressing the window over which its belief could be seen to bloom. Let’s give the **root** more dynamic range. This pass increments the tree level, so context length becomes 16, and we deal with **fourth** latent level. Other HRM parameters remain the same.¹⁵ We consider ten **non-isomorphic** grammars and three master seeds again. We vary model depth $n_{\text{layer}} \in \{2, 3\}$. Overall we make **60 runs**¹⁶. These run close 0.96–0.99 of the uniform→Bayes loss gap (mean 0.99) and are reported in full in Table 7.

- **All earlier findings replicated at $L = 4$.**

Linear decodability growth held (every level is read off well above the shuffled baseline of ≈ -0.04); the belief sharpens with context (ground truth root posterior entropy falls $1.86 \rightarrow 0.00$ monotonically over the 16 positions, Figure 18); belief **accumulates across layers** (root probe R^2 rises $-0.01 \rightarrow 0.07 \rightarrow 0.19$ for $n_{\text{layer}} = 2$ and $-0.01 \rightarrow 0.06 \rightarrow 0.17 \rightarrow 0.28$ for $n_{\text{layer}} = 3$, the deeper model will reach a higher readout, Figure 19); and causal steering still works (Figure 20).

Root decodability becomes even weaker at $L = 4$ than at $L = 3$: the $n_{\text{layer}} = 2$ family mean falls to 0.19 (from 0.35 at $L = 3$), and even adding a third layer lifts it only to 0.28, still below the $L = 3$ value of 0.35.

The extra depth gives the **root specifically** room to bloom. At $L = 3$ the root collapsed to certainty within about three tokens, so its own belief barely had a window to expand (the $L = 3$ blooming we showed earlier is dominated by the local latents); the longer 16-position $L = 4$ context stretches that window out. Over it the ground truth **root** posterior entropy decreases smoothly and monotonically from 1.86 nats to 0, and the root belief readout radius grows overall from 0.18 to 0.50 (Figure 18). The radius growth is noisier than the local latents’ — it wobbles in the back half — but the root now visibly sharpens with context rather than snapping to certainty.

The per-level gradient takes the inverted U-shape (Figure 17). Family-mean probe R^2 is $L_0 = 0.24$, $L_1 = 0.37$, $L_2 = 0.36$, $L_3 = 0.30$: it **rises** from the root to the mid-levels and then **falls** back toward the

¹⁵($s = 2, v = 8, m = 2$, uniform sampling, unambiguous trees)

¹⁶`run_depth4.py`, writing `results/depth4/g{NN}_L{n}_d128_h4_t4000_s{S}/`, a separate root so the depth-agnostic dir names never collide with previous passes)

unclear
what is
written here

leaf-parents. Only 6 of 60 runs show the strict $L_0 < L_1 < L_2 < L_3$ ordering. Root remains the weakest decodable latent.

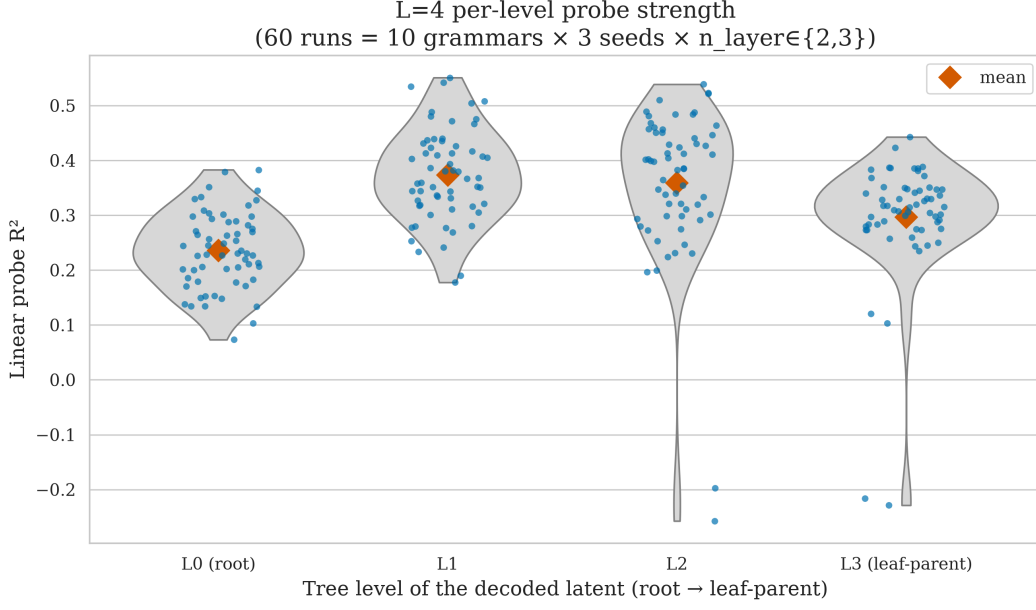


Figure 17: $L = 4$ per-level probe strength across the family. Each point is one of 60 runs (10 grammars $\times$ 3 seeds $\times$ $n_{\text{layer}} \in \{2, 3\}$); violins show the per-level R^2 distribution, diamonds the means. The gradient is an **inverted-U** — the mid-levels (L_1, L_2) decode strongest, the root (L_0) weakest, the leaf-parents (L_3) intermediate — not the pre-registered monotone ladder.

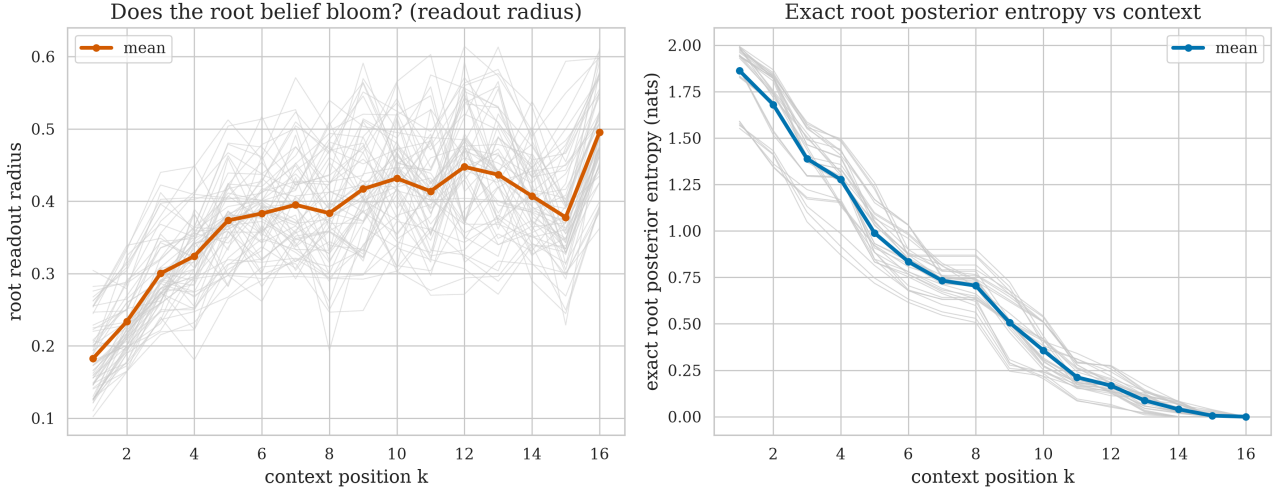


Figure 18: The root **does** bloom at $L = 4$. **Left:** the root belief readout radius grows with context position k over the 16-position window (grey: 60 runs; orange: mean), though non-monotonically in the back half. **Right:** the ground truth root posterior entropy decreases smoothly and monotonically from 1.86 nats to 0.

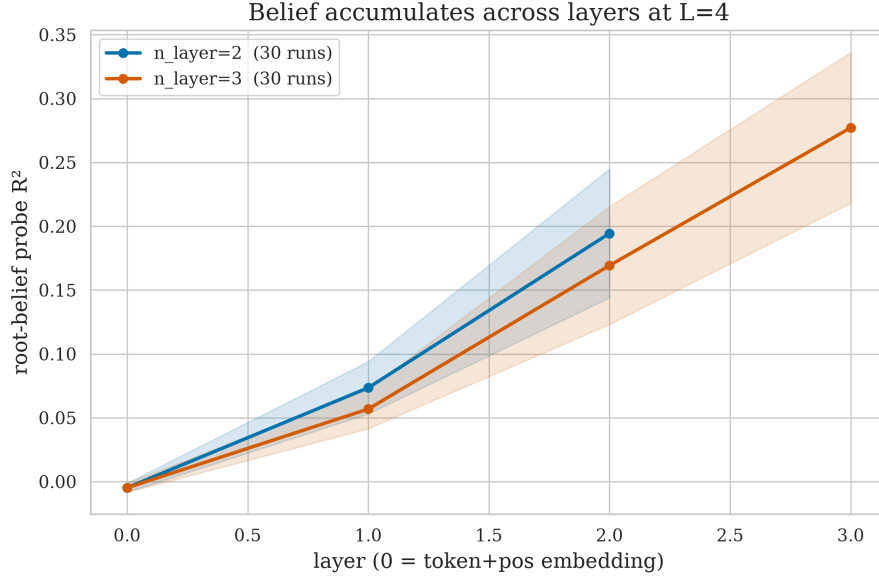


Figure 19: Belief accumulates across layers at $L = 4$. Root-belief probe R^2 vs residual index, one line per n_{layer} (mean $\pm$ SD over 30 runs each). Both archs rise monotonically from the embedding to the readout; the 3-layer model reaches a higher endpoint (0.28 vs 0.19).

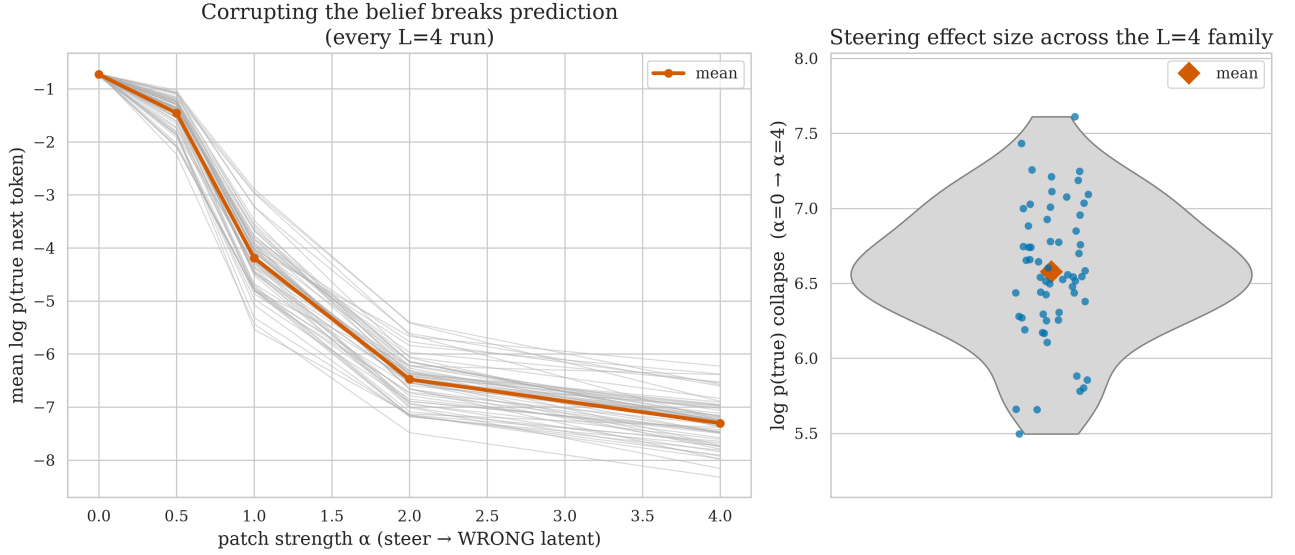


Figure 20: Causal steering replicates at $L = 4$. **Left:** steering the layer-1 residual toward a **wrong** latent collapses the true next token’s log-probability in every run (grey), mean in orange. **Right:** the collapse magnitude (grey violin, orange diamond = mean) for $\alpha=\{0 \rightarrow 4\}$ across the 60 runs: 6.6 ± 0.5 nats.

2. Appendix: Root reconstruction diagnostics

The root-posterior probe is a regression target, but it is also useful to ask the harsher classification question: does the largest coordinate of the linear belief readout recover the sampled root class? Across 400 probe examples and 8 context positions (3200 example-position points), the answer is yes for 1565 points (48.91%). This is well above random guessing, but random guessing is only $1/8 = 12.5\%$ because there are eight root classes; the relevant baseline is not 50%.

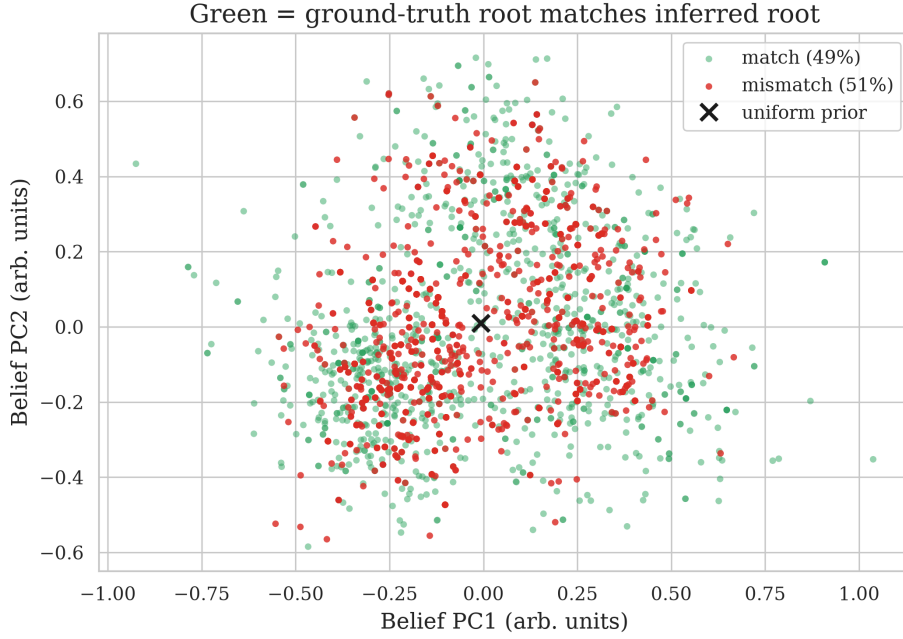


Figure 21: Root reconstruction from the linear belief readout. Green points are example-position pairs where the readout’s argmax matches the ground-truth root; red points are mismatches. The overall match rate is 48.91%, compared with a random-guessing baseline of 12.5% (not 50%) for eight root classes.

Restricting to positions where nearly all evidence is visible makes the diagnostic sharper. With seven of eight symbols observed ($k = 6$), the ground truth Bayesian posterior’s MAP root already matches the sampled root in 95.00% of examples, while the linear readout matches in 62.25%. With all eight symbols observed ($k = 7$), the ground truth posterior is deterministic, but the readout still reaches only 70.50%. Thus early ambiguity explains part, but not all, of the fuzzy root reconstruction.

Subset	Prefix length	Readout match	Exact MAP match
All example-position points	1-8	$\frac{1565}{3200} = 48.91\%$	—
All but last symbol	7	$\frac{249}{400} = 62.25\%$	$\frac{380}{400} = 95.00\%$
Full sequence	8	$\frac{282}{400} = 70.50\%$	$\frac{400}{400} = 100.00\%$

3. Appendix: The rule table as a graph

Table 1 lists the frozen grammar as text, three columns of or-separated child pairs. The same information is easier to scan as a graph: four columns of eight nodes (root, level 1, level 2, leaves), each parent’s two rules drawn as two distinctly colored pairs of arrows into the level below. No edge skips a level, which is the “no cross-level ambiguity” property by construction.

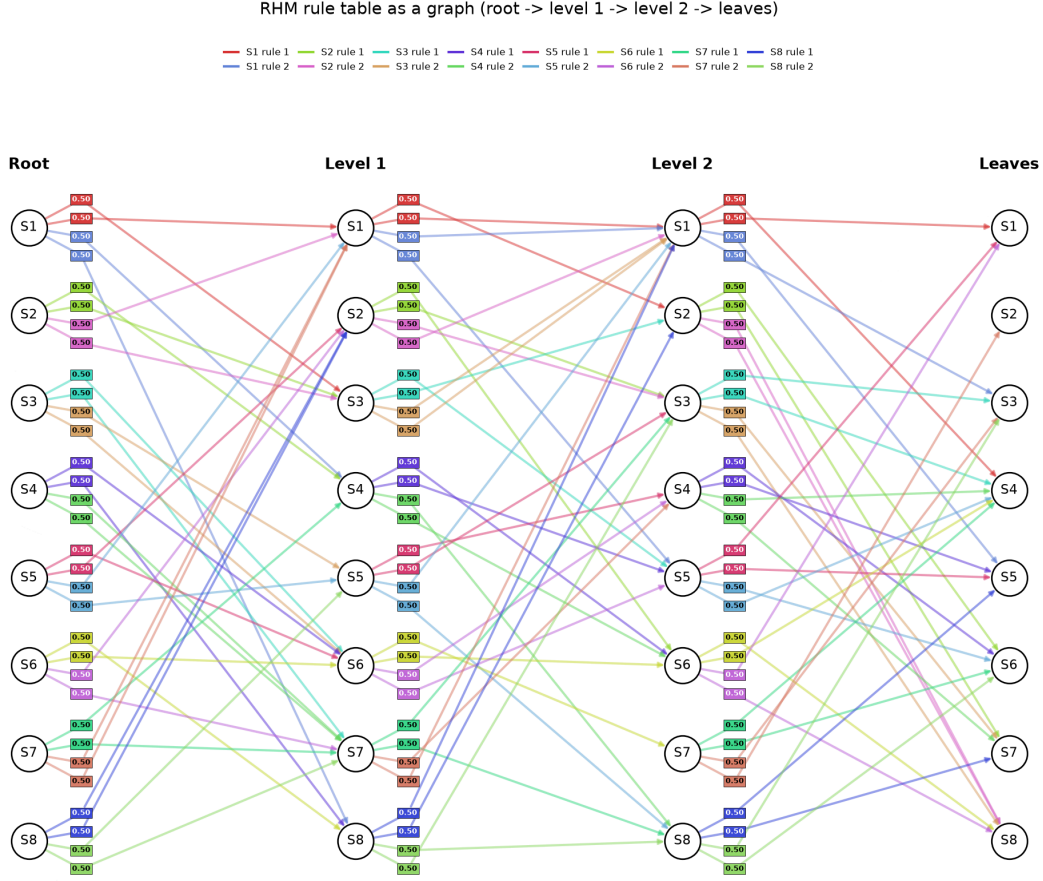


Figure 22: The frozen rule table of Table 1 redrawn as a DAG. Each of the eight symbols S1-S8 appears once per level; its two production rules are drawn in two distinct colors (one per rule), chosen so that neither a node’s own two colors nor neighboring nodes’ colors are easily confused. This grammar is **unambiguous**: no two rules (across all parents at a level) share a child pair, so every child pair has a unique parent and no leaf substring admits two readings — see Appendix 4 for a grammar where ambiguity is deliberately introduced and several parents’ edges converge on a shared child pair.

4. Appendix: Skewed and ambiguous example grammars

The main-text grammar (Table 1) has uniform rule-choice probabilities and no shared child-tuples across parents. To illustrate the two knobs used by the non-collapse sweep (Appendix 8), this appendix shows two concrete example grammars, restored exactly from their saved `grammar.npz`: the draw-0, skew=high grammar (`results/noncollapse/skhigh_am0_g00_L2_d128_h4_t4000_s0`) and the draw-0, $\rho = 0.6$ grammar (`results/noncollapse/sknone_am0.6_g00_L2_d128_h4_t4000_s0`). Both tables and graphs are generated directly from the saved arrays, not hand-transcribed.

Parent	Top expansion	Middle expansion	Bottom expansion
S1	$0.995 \cdot [S3, S1]$ or $0.00467 \cdot [S4, S8]$	$0.0978 \cdot [S1, S5]$ or $0.902 \cdot [S6, S3]$	$0.0000894 \cdot [S5, S8]$ or $1.000 \cdot [S7, S2]$
S2	$0.000000134 \cdot [S4, S3]$ or $1.000 \cdot [S1, S3]$	$0.0000129 \cdot [S8, S2]$ or $1.000 \cdot [S6, S1]$	$0.584 \cdot [S6, S6]$ or $0.416 \cdot [S4, S1]$
S3	$0.368 \cdot [S6, S7]$ or $0.632 \cdot [S5, S6]$	$0.991 \cdot [S7, S6]$ or $0.00869 \cdot [S5, S3]$	$0.0368 \cdot [S3, S7]$ or $0.963 \cdot [S4, S6]$
S4	$1.000 \cdot [S6, S8]$ or $0.000291 \cdot [S7, S7]$	$0.404 \cdot [S4, S5]$ or $0.596 \cdot [S3, S5]$	$0.822 \cdot [S1, S3]$ or $0.178 \cdot [S6, S7]$
S5	$0.597 \cdot [S6, S2]$ or $0.403 \cdot [S1, S5]$	$0.584 \cdot [S1, S4]$ or $0.416 \cdot [S2, S5]$	$0.933 \cdot [S5, S5]$ or $0.0665 \cdot [S6, S1]$

S6	$0.948 \cdot [S8, S6]$ or $0.0519 \cdot [S2, S7]$	$0.840 \cdot [S7, S1]$ or $0.160 \cdot [S2, S1]$	$0.999 \cdot [S2, S4]$ or $0.000881 \cdot [S2, S6]$
S7	$0.889 \cdot [S4, S7]$ or $0.111 \cdot [S1, S1]$	$0.174 \cdot [S5, S7]$ or $0.826 \cdot [S3, S1]$	$0.983 \cdot [S7, S6]$ or $0.0168 \cdot [S8, S5]$
S8	$0.993 \cdot [S2, S2]$ or $0.00687 \cdot [S5, S7]$	$0.998 \cdot [S3, S4]$ or $0.00178 \cdot [S2, S2]$	$0.409 \cdot [S1, S7]$ or $0.591 \cdot [S2, S1]$

Table 3: Skewed example grammar (skew=high, Dirichlet $\alpha=0.2$, $\rho=0$). Each cell is prefixed by the exact rule-choice probability from the saved grammar.npz; several parents are near-deterministic (one rule near 1).

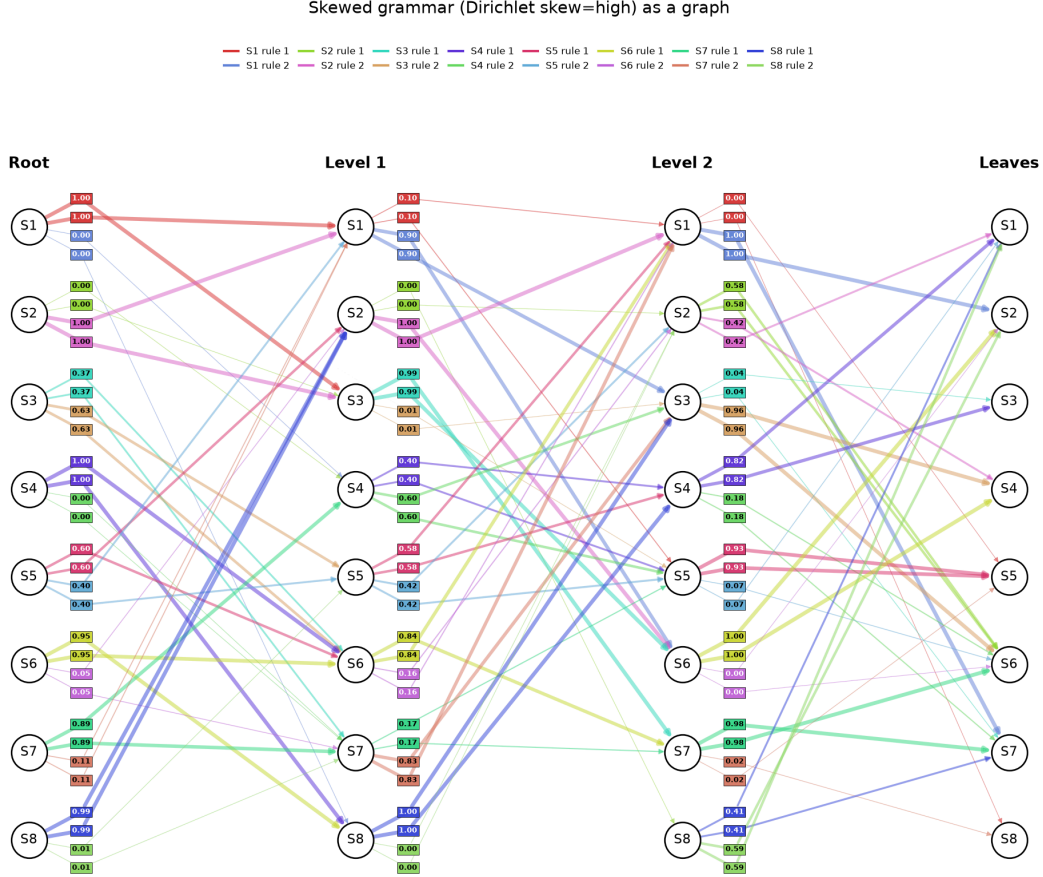


Figure 23: The skewed grammar as a DAG. Each of a parent’s four outgoing edges carries a small colored square near the parent holding its rule’s exact choice probability ($\alpha = 0.2$ symmetric Dirichlet draw; also in the table cells) — a rule’s two edges share colour and value, and the four squares are stacked at fixed symmetric slots (two above, two below) so they stay aligned and never overlap. Edge linewidth is proportional to the probability, so a near-deterministic parent (one rule ≈ 1) shows thick edges with 1.00 squares for its dominant rule and faint, thin edges with 0.00 squares for the near-zero alternative.

Parent	Top expansion	Middle expansion	Bottom expansion
S1	$[S7, S7]$ or $[S5, S7]$	$[S7, S1]$ or $[S6, S1]$	$[S8, S5]$ or $[S7, S6]$
S2	$[S4, S3]$ or $[S6, S2]$	$[S3, S5]$ or $[S8, S6]$	$[S8, S5]$ or $[S7, S6]$
S3	$[S6, S7]$ or $[S5, S7]$	$[S8, S1]$ or $[S7, S1]$	$[S7, S5]$ or $[S8, S5]$
S4	$[S5, S7]$ or $[S1, S1]$	$[S6, S6]$ or $[S3, S7]$	$[S7, S6]$ or $[S1, S5]$
S5	$[S6, S2]$ or $[S4, S7]$	$[S6, S6]$ or $[S5, S4]$	$[S2, S5]$ or $[S5, S1]$
S6	$[S8, S6]$ or $[S5, S7]$	$[S4, S2]$ or $[S7, S1]$	$[S8, S5]$ or $[S5, S1]$
S7	$[S4, S7]$ or $[S1, S1]$	$[S6, S6]$ or $[S6, S1]$	$[S8, S5]$ or $[S4, S4]$
S8	$[S4, S7]$ or $[S1, S1]$	$[S6, S6]$ or $[S4, S7]$	$[S1, S5]$ or $[S4, S4]$

Table 4: Ambiguous example grammar (skew=none, rho=0.6). Rule-choice probabilities are uniform (1/m), so cells show plain child pairs; pairs marked in bold are shared child-tuples produced by more than one (parent, rule) at that level.

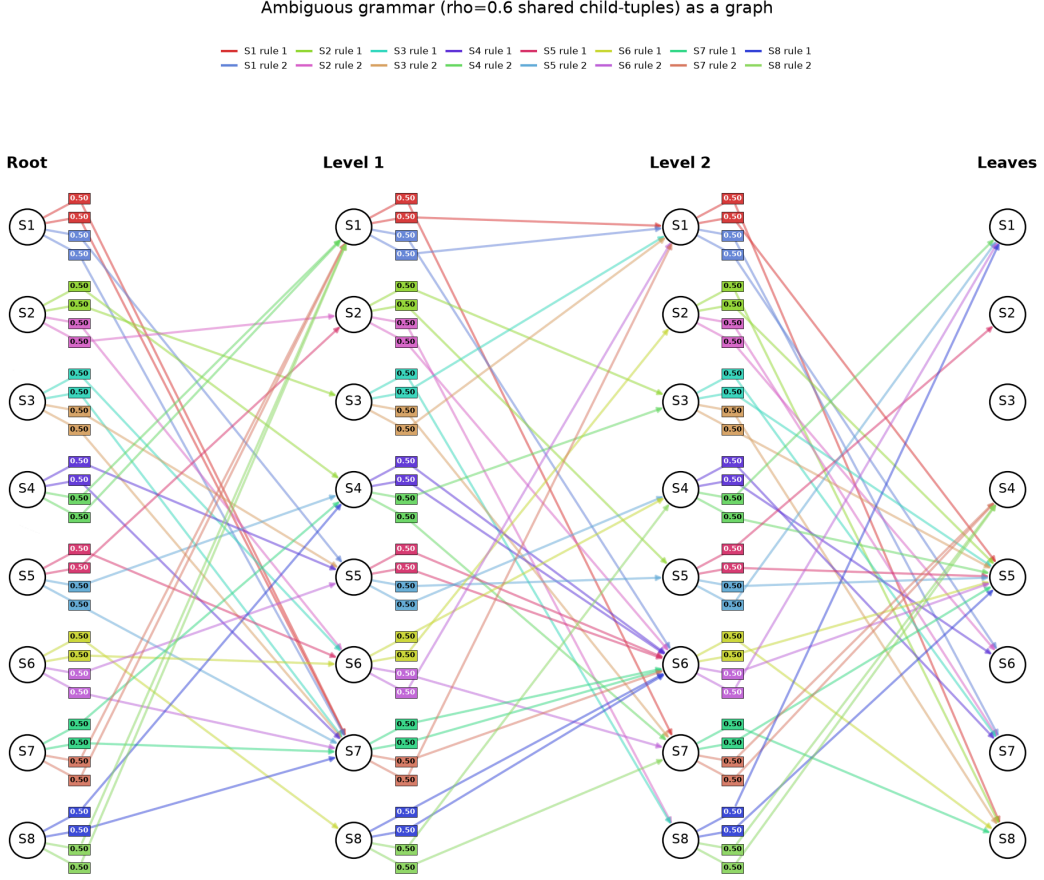


Figure 24: The ambiguous grammar as a DAG. Rule-choice probabilities are uniform, but child-tuples are shared across parents at rate $\rho = 0.6$, so several parents' rules point at the same child pair (bold cells in the table; edges converging on the same node in the graph) – the many-to-one structure that makes the belief non-collapsing.

5. Appendix: Per-run grammar-sweep sanity table

Every run in the grammar sweep, no filtering. `grammar` indexes the rule-table draw (`Grammar.random(seed=grammar)`); `seed` is the master replicate seed; the architecture is pinned. `test_ce` is the held-out next-token cross-entropy; `loss_gap_closed` is the fraction of the uniform→Bayes gap closed (sanity covariate, not a gate); `root_r2` and `deepest_r2` are the level- L_0 and mean level- L_2 probe R^2 . Loaded directly from `figures/sweep_sanity.csv`.

grammar	seed	n_layer	n_embd	n_head	test_ce	loss_gap_closed	root_r2	deepest_r2
0	0	2	128	4	0.931	0.848	0.242	0.498
0	1	2	128	4	0.891	0.878	0.243	0.497
0	2	2	128	4	0.918	0.858	0.281	0.532
1	0	2	128	4	0.847	0.906	0.375	0.556
1	1	2	128	4	0.812	0.932	0.479	0.616
1	2	2	128	4	0.845	0.908	0.353	0.496
2	0	2	128	4	0.905	0.886	0.402	-0.061
2	1	2	128	4	0.858	0.922	0.454	0.479
2	2	2	128	4	0.866	0.916	0.354	0.46
3	0	2	128	4	0.845	0.91	0.336	0.253
3	1	2	128	4	0.912	0.86	0.333	0.611
3	2	2	128	4	0.92	0.854	0.311	0.607

grammar	seed	n_layer	n_embd	n_head	test_ce	loss_gap_closed	root_r2	deepest_r2
4	0	2	128	4	0.925	0.876	0.442	0.505
4	1	2	128	4	0.884	0.907	0.416	0.497
4	2	2	128	4	0.909	0.888	0.29	0.442
5	0	2	128	4	0.853	0.911	0.25	0.461
5	1	2	128	4	0.841	0.92	0.277	0.482
5	2	2	128	4	0.848	0.914	0.334	0.493
6	0	2	128	4	0.868	0.896	0.351	0.523
6	1	2	128	4	0.882	0.885	0.4	0.553
6	2	2	128	4	0.836	0.919	0.312	0.523
7	0	2	128	4	0.964	0.82	0.298	0.441
7	1	2	128	4	0.909	0.861	0.256	0.359
7	2	2	128	4	0.892	0.874	0.326	0.436
8	0	2	128	4	0.897	0.863	0.338	0.524
8	1	2	128	4	0.838	0.906	0.359	0.532
8	2	2	128	4	0.868	0.884	0.327	0.532
9	0	2	128	4	0.899	0.894	0.411	0.557
9	1	2	128	4	0.988	0.827	0.394	0.565
9	2	2	128	4	0.87	0.916	0.438	0.604

Table 5: All 30 grammar-sweep runs (10 grammars $\times$ 3 seeds), pinned architecture, full 4000-step budget. No convergence filtering.

6. Appendix: Per-run architecture-sweep sanity table

Every run in the architecture sweep, no filtering — all 99 (11 one-axis-at-a-time capacity configs $\times$ 3 grammars $\times$ 3 seeds). `n_layer`, `n_embd`, `n_head`, `steps` give the capacity config; `grammar` and `seed` the replicate. `test_ce` is the held-out next-token cross-entropy; `loss_gap_closed` the fraction of the uniform $\rightarrow$ Bayes gap closed (covariate, not a gate); `root_r2` and `deepest_r2` the level- L_0 and mean level- L_2 probe R^2 . The deliberately-underpowered small models (e.g. $n_{\text{embd}} = 16$) are included. Loaded directly from `figures/arch_sanity.csv`.

grammar	seed	n_layer	n_embd	n_head	steps	test_ce	loss_gap_closed	root_r2	deepest_r2
0	0	1	128	4	4000	0.974	0.816	0.136	0.374
0	1	1	128	4	4000	0.933	0.847	0.131	0.403
0	2	1	128	4	4000	1.028	0.776	0.114	0.354
1	0	1	128	4	4000	0.913	0.858	0.271	0.475
1	1	1	128	4	4000	0.873	0.887	0.268	0.483
1	2	1	128	4	4000	0.854	0.901	0.245	0.421
2	0	1	128	4	4000	0.92	0.875	0.292	-0.138
2	1	1	128	4	4000	0.872	0.912	0.248	0.344
2	2	1	128	4	4000	0.911	0.882	0.298	0.382
0	0	2	16	4	4000	1.065	0.749	0.022	0.172
0	1	2	16	4	4000	1.048	0.762	0.045	0.203
0	2	2	16	4	4000	1.038	0.769	0.032	0.183
1	0	2	16	4	4000	0.966	0.819	0.065	0.233
1	1	2	16	4	4000	0.951	0.83	0.079	0.216
1	2	2	16	4	4000	0.921	0.852	0.073	0.222
2	0	2	16	4	4000	0.943	0.857	0.097	-0.115
2	1	2	16	4	4000	0.973	0.835	0.08	0.096
2	2	2	16	4	4000	0.934	0.864	0.107	0.12
0	0	2	64	4	4000	1.107	0.718	0.143	0.329
0	1	2	64	4	4000	1.236	0.623	0.127	0.358
0	2	2	64	4	4000	1.225	0.631	0.151	0.351
1	0	2	64	4	4000	1.05	0.757	0.258	0.397
1	1	2	64	4	4000	1.182	0.66	0.225	0.381
1	2	2	64	4	4000	1.084	0.732	0.212	0.343
2	0	2	64	4	4000	1.182	0.677	0.249	-0.121
2	1	2	64	4	4000	1.122	0.722	0.283	0.304
2	2	2	64	4	4000	1.103	0.737	0.258	0.296

grammar	seed	n_layer	n_embd	n_head	steps	test_ce	loss_gap_closed	root_r2	deepest_r2
0	0	2	128	1	4000	0.919	0.857	0.205	0.458
0	1	2	128	1	4000	0.936	0.844	0.165	0.468
0	2	2	128	1	4000	1.07	0.745	0.155	0.432
1	0	2	128	1	4000	0.931	0.844	0.266	0.527
1	1	2	128	1	4000	0.943	0.836	0.274	0.512
1	2	2	128	1	4000	0.931	0.845	0.287	0.495
2	0	2	128	1	4000	0.916	0.878	0.369	0.091
2	1	2	128	1	4000	0.937	0.862	0.322	0.449
2	2	2	128	1	4000	1.02	0.799	0.247	0.371
0	0	2	128	2	4000	0.891	0.878	0.288	0.508
0	1	2	128	2	4000	0.871	0.893	0.269	0.594
0	2	2	128	2	4000	0.899	0.871	0.17	0.465
1	0	2	128	2	4000	0.856	0.9	0.449	0.585
1	1	2	128	2	4000	0.81	0.934	0.432	0.619
1	2	2	128	2	4000	0.867	0.892	0.344	0.495
2	0	2	128	2	4000	1.023	0.797	0.37	-0.19
2	1	2	128	2	4000	0.877	0.908	0.353	0.453
2	2	2	128	2	4000	0.859	0.921	0.365	0.477
0	0	2	128	4	4000	1.019	0.783	0.182	0.409
0	1	2	128	4	4000	0.964	0.824	0.214	0.498
0	2	2	128	4	4000	0.877	0.888	0.301	0.515
1	0	2	128	4	4000	0.89	0.875	0.467	0.576
1	1	2	128	4	4000	0.832	0.918	0.396	0.592
1	2	2	128	4	4000	0.819	0.927	0.405	0.54
2	0	2	128	4	4000	0.837	0.938	0.418	-0.028
2	1	2	128	4	4000	0.935	0.864	0.37	0.404
2	2	2	128	4	4000	0.883	0.903	0.426	0.464
0	0	2	128	4	16000	1.307	0.571	0.304	0.477
0	1	2	128	4	16000	1.17	0.671	0.298	0.566
0	2	2	128	4	16000	1.278	0.592	0.304	0.517
1	0	2	128	4	16000	1.086	0.73	0.406	0.558
1	1	2	128	4	16000	1.074	0.739	0.442	0.603
1	2	2	128	4	16000	0.972	0.814	0.474	0.583
2	0	2	128	4	16000	0.984	0.827	0.497	-0.171
2	1	2	128	4	16000	1.096	0.742	0.478	0.552
2	2	2	128	4	16000	0.895	0.894	0.448	0.506
0	0	2	128	8	4000	1.046	0.763	0.253	0.461
0	1	2	128	8	4000	1.016	0.786	0.272	0.532
0	2	2	128	8	4000	0.917	0.859	0.269	0.521
1	0	2	128	8	4000	0.936	0.841	0.456	0.558
1	1	2	128	8	4000	0.948	0.832	0.369	0.527
1	2	2	128	8	4000	0.82	0.926	0.369	0.488
2	0	2	128	8	4000	0.885	0.901	0.431	-0.051
2	1	2	128	8	4000	0.903	0.888	0.405	0.456
2	2	2	128	8	4000	0.896	0.893	0.439	0.488
0	0	2	256	4	4000	0.891	0.877	0.292	0.635
0	1	2	256	4	4000	0.887	0.88	0.21	0.546
0	2	2	256	4	4000	0.833	0.92	0.329	0.673
1	0	2	256	4	4000	0.86	0.897	0.518	0.693
1	1	2	256	4	4000	0.828	0.92	0.646	0.775
1	2	2	256	4	4000	0.803	0.939	0.627	0.753
2	0	2	256	4	4000	0.912	0.881	0.405	0.167
2	1	2	256	4	4000	0.895	0.894	0.344	0.489
2	2	2	256	4	4000	0.89	0.898	0.54	0.609
0	0	3	128	4	4000	0.937	0.844	0.297	0.514
0	1	3	128	4	4000	0.867	0.895	0.319	0.581
0	2	3	128	4	4000	0.883	0.883	0.3	0.513
1	0	3	128	4	4000	0.825	0.923	0.443	0.562
1	1	3	128	4	4000	0.882	0.881	0.398	0.548
1	2	3	128	4	4000	0.803	0.939	0.36	0.522
2	0	3	128	4	4000	0.855	0.924	0.414	-0.137
2	1	3	128	4	4000	0.861	0.92	0.47	0.506
2	2	3	128	4	4000	0.865	0.916	0.463	0.508
0	0	4	128	4	4000	0.884	0.883	0.311	0.525

grammar	seed	n_layer	n_embd	n_head	steps	test_ce	loss_gap_closed	root_r2	deepest_r2
0	1	4	128	4	4000	0.83	0.922	0.272	0.564
0	2	4	128	4	4000	0.836	0.918	0.288	0.557
1	0	4	128	4	4000	0.803	0.939	0.502	0.602
1	1	4	128	4	4000	0.838	0.913	0.461	0.61
1	2	4	128	4	4000	0.804	0.938	0.442	0.529
2	0	4	128	4	4000	0.853	0.925	0.46	-0.082
2	1	4	128	4	4000	0.832	0.941	0.5	0.537
2	2	4	128	4	4000	0.83	0.943	0.497	0.508

Table 6: All 99 architecture-sweep runs (11 one-axis-at-a-time capacity configs $\times$ 3 grammars $\times$ 3 seeds). No convergence filtering; small models are expected to underfit.

7. Appendix: Per-run $L = 4$ depth-sweep sanity table

Every run in the $L = 4$ depth sweep, no filtering — all 60 (10 grammars $\times$ 3 seeds $\times$ $n_{\text{layer}} \in \{2, 3\}$). **grammar** and **seed** index the rule-table draw and master replicate seed; **n_layer** is the only varied architecture axis ($n_{\text{embd}} = 128$, $n_{\text{head}} = 4$, 4000 steps throughout). **test_ce** is the held-out next-token cross-entropy; **loss_gap_closed** the fraction of the uniform $\rightarrow$ Bayes gap closed (covariate, not a gate); **root_r2** and **deepest_r2** the level- L_0 and mean leaf-parent- L_3 probe R^2 . Loaded directly from **figures/depth4_sanity.csv**.

grammar	seed	n_layer	test_ce	loss_gap_closed	root_r2	deepest_r2
0	0	2	0.716	0.992	0.207	0.32
0	0	3	0.716	0.991	0.23	0.35
0	1	2	0.715	0.992	0.178	0.345
0	1	3	0.719	0.99	0.263	0.386
0	2	2	0.728	0.983	0.148	0.307
0	2	3	0.711	0.995	0.276	0.33
1	0	2	0.728	0.982	0.227	0.284
1	0	3	0.713	0.993	0.282	0.298
1	1	2	0.715	0.992	0.227	0.28
1	1	3	0.718	0.989	0.288	0.341
1	2	2	0.722	0.987	0.179	0.273
1	2	3	0.715	0.992	0.298	0.336
2	0	2	0.724	0.987	0.203	0.317
2	0	3	0.726	0.986	0.271	0.372
2	1	2	0.727	0.985	0.171	0.304
2	1	3	0.718	0.992	0.301	0.384
2	2	2	0.725	0.986	0.149	0.315
2	2	3	0.72	0.99	0.318	0.386
3	0	2	0.74	0.986	0.231	0.25
3	0	3	0.732	0.992	0.304	0.235
3	1	2	0.783	0.955	0.22	0.257
3	1	3	0.73	0.994	0.351	0.244
3	2	2	0.75	0.979	0.211	0.273
3	2	3	0.735	0.99	0.345	0.301
4	0	2	0.73	0.987	0.183	0.381
4	0	3	0.722	0.992	0.33	0.423
4	1	2	0.723	0.992	0.171	0.347
4	1	3	0.731	0.986	0.2	0.368
4	2	2	0.722	0.992	0.244	0.388
4	2	3	0.721	0.993	0.308	0.443
5	0	2	0.745	0.978	0.073	0.273
5	0	3	0.732	0.988	0.134	0.288
5	1	2	0.741	0.981	0.103	0.317
5	1	3	0.728	0.991	0.205	0.33
5	2	2	0.748	0.976	0.134	0.283
5	2	3	0.743	0.98	0.153	0.306
6	0	2	0.718	0.992	0.213	0.31
6	0	3	0.72	0.991	0.29	0.386
6	1	2	0.731	0.983	0.134	0.29
6	1	3	0.731	0.982	0.206	0.351
6	2	2	0.723	0.988	0.138	0.299

grammar	seed	n_layer	test_ce	loss_gap_closed	root_r2	deepest_r2
6	2	3	0.717	0.993	0.266	0.34
7	0	2	0.732	0.987	0.254	0.259
7	0	3	0.744	0.978	0.228	0.274
7	1	2	0.729	0.989	0.227	0.103
7	1	3	0.762	0.965	0.249	0.121
7	2	2	0.732	0.987	0.186	-0.229
7	2	3	0.737	0.983	0.245	-0.216
8	0	2	0.718	0.988	0.202	0.292
8	0	3	0.714	0.99	0.257	0.33
8	1	2	0.727	0.981	0.236	0.276
8	1	3	0.713	0.991	0.294	0.315
8	2	2	0.732	0.978	0.152	0.245
8	2	3	0.718	0.988	0.327	0.347
9	0	2	0.727	0.986	0.27	0.298
9	0	3	0.727	0.986	0.383	0.348
9	1	2	0.721	0.991	0.264	0.328
9	1	3	0.716	0.994	0.379	0.387
9	2	2	0.717	0.993	0.298	0.326
9	2	3	0.724	0.988	0.333	0.351

Table 7: All 60 $L = 4$ depth-sweep runs ($10 \text{ grammars} \times 3 \text{ seeds} \times n_{\text{layer}} \in \{2, 3\}$). No convergence filtering. **deepest_r2** is the mean over the eight leaf-parent (L_3) nodes.

8. Appendix: Per-run non-collapse sweep sanity table

Every run in the non-collapse sweep, no filtering — all 27 ($3 \text{ skew} \times 3 \text{ ambiguity} \times 3 \text{ rule-table draws}$), at the pinned arch ($n_{\text{layer}} = 2, n_{\text{embd}} = 128, n_{\text{head}} = 4, 4000 \text{ steps}$). **bayes_floor** is the exact weighted Bayes-optimal next-token entropy; **loss_gap_closed** the fraction of the uniform→Bayes gap closed (covariate); **root_r2/mid_r2/low_r2** the level- $L_0/L_1/L_2$ probe R^2 (shuffled baseline **shuffled_r2**); **k8_entropy** the headline mean $k = 8$ root posterior entropy; **eff_dim/hull_area** the reachable-set descriptors. Loaded directly from **figures/noncollapse_sanity.csv**.

skew	ambiguity	draw	test_ce	bayes_floor	loss_gap_closed	root_r2	mid_r2	low_r2	shuffled_r2	k8_entropy	eff_dim	hull_area
high	0.0000	0	1.1915	0.2818	0.4939	0.2369	0.3522	0.4214	-0.0868	0.0000	6.9652	0.6465
high	0.0000	1	0.8751	0.3953	0.7151	0.2492	0.3554	0.4202	-0.0628	0.0000	6.9652	0.6465
high	0.0000	2	1.0670	0.2532	0.5544	0.2983	0.1920	0.3542	-0.0782	0.0000	6.9652	0.6465
high	0.3000	0	0.9041	0.2118	0.6293	0.3115	0.3577	0.5997	-0.0447	0.2655	5.6327	0.5984
high	0.3000	1	0.3135	0.1622	0.9211	0.4511	0.5554	0.5639	-0.0808	0.3314	4.9094	0.6720
high	0.3000	2	0.6246	0.2468	0.7938	0.3783	0.4539	0.5344	-0.1115	0.4441	4.7952	0.6780
high	0.6000	0	0.3267	0.1826	0.9241	0.5202	0.6696	0.6840	-0.0675	0.5852	3.7205	0.6501
high	0.6000	1	0.2625	0.2101	0.9720	0.5496	0.6274	0.8281	-0.0704	0.6123	3.6509	0.7863
high	0.6000	2	0.2848	0.2101	0.9601	0.5612	0.7059	0.7040	-0.1028	0.4243	2.8199	0.6248
mid	0.0000	0	0.8921	0.5366	0.7696	0.3287	0.4008	0.3964	-0.0608	0.0000	6.9652	0.6465
mid	0.0000	1	1.0199	0.5468	0.6913	0.3799	0.5235	0.5137	-0.0439	0.0000	6.9652	0.6465
mid	0.0000	2	1.1025	0.5426	0.6357	0.3549	0.3549	-7.4713	-0.0763	0.0000	6.9652	0.6465
mid	0.3000	0	0.6272	0.4867	0.9118	0.4159	0.4348	0.5595	-0.0552	0.6537	4.8768	0.5921
mid	0.3000	1	0.6823	0.4995	0.8843	0.4557	0.6463	0.6656	-0.0553	0.3870	5.8288	0.6014
mid	0.3000	2	0.6325	0.5326	0.9354	0.3626	0.4213	0.6025	-0.0270	0.7169	5.2545	0.7321
mid	0.6000	0	0.5376	0.4261	0.9326	0.5044	0.6875	0.8028	-0.0715	1.1605	4.3417	0.5354
mid	0.6000	1	0.6313	0.5088	0.9220	0.3874	0.5424	0.6770	-0.0920	0.9423	3.8371	0.6337
mid	0.6000	2	0.5529	0.4420	0.9323	0.4711	0.7290	0.7901	-0.1003	1.0470	4.0308	0.4432
none	0.0000	0	0.8900	0.7253	0.8784	0.3002	0.4092	0.5039	-0.0568	0.0000	6.9652	0.6465
none	0.0000	1	0.8494	0.7199	0.9047	0.3578	0.5082	0.5504	-0.0638	0.0000	6.9652	0.6465
none	0.0000	2	0.8447	0.7544	0.9319	0.4262	0.4582	0.0083	-0.0640	0.0000	6.9652	0.6465
none	0.3000	0	0.6930	0.6221	0.9513	0.4455	0.4894	0.6512	-0.0439	0.7598	5.3126	0.6421
none	0.3000	1	0.7216	0.6509	0.9505	0.4274	0.6678	0.6417	-0.0553	0.6576	5.5352	0.5416
none	0.3000	2	0.7397	0.6660	0.9479	0.3771	0.5106	0.5869	-0.0733	0.7003	5.4769	0.5762
none	0.6000	0	0.5011	0.4860	0.9905	0.5640	0.6747	0.7615	-0.1606	1.3969	3.7241	0.6400
none	0.6000	1	0.5658	0.5448	0.9863	0.6170	0.7852	0.9018	-0.0745	1.3457	3.3016	0.5520
none	0.6000	2	0.5328	0.4976	0.9778	0.4220	0.6758	0.7826	-0.0481	1.4068	2.7883	0.4318

Table 8: All 27 non-collapse runs ($3 \text{ skew} \times 3 \text{ ambiguity} \times 3 \text{ draws}$). No filtering. **k8_entropy** ≈ 0 exactly when $\rho = 0$ regardless of skew.

9. Appendix: Reproducing every number and figure

This appendix answers, for every number and figure cited above, exactly how to get it again: which script to run, with which flags, and which file on disk currently holds the value. Pipeline shape, for orientation: **rh.m.py** (grammar + exact belief propagation, library only) → **train.py** (one training run)

→ `analyze.py` (probe + causal steering for one run’s artifacts) → `sweep.py` (one sweep **cell** = train + analyze, deterministic dir + `config.json` manifest; usable standalone or under `wandb agent`) → `run_arch.py` / `run_depth4.py` / `run_noncollapse.py` (loop `sweep.py --no-wandb` over a grid) → the aggregator scripts in `paper-typst/figures/scripts/` (one per pass) that glob `results/{pass}/*/` `{config.json,analysis.json}` and emit the paper’s PNGs, `*_sanity.csv`, and (for passes 4-5) scored-prereg JSON. Figure generation is **not** wired into `ninja` — only the Typst compile is (`build.ninja: typst compile --ignore-system-fonts paper-typst/main.typ paper-typst/main.pdf`); regenerating a figure is always a separate, manual `uv run python paper-typst/figures/scripts/{name}.py`.

Load-bearing caveat. The pass-1 canonical run behind `artifacts/` (and hence `results/analysis.json`, `results/root_reconstruction.json`, and Figure 2–Figure 7) was trained **before** model-init seeding was added to the codebase (added in the pass-2 refactor; see `WORKLOG.md`, 2026-06-29T19:35). Re-running the commands below reproduces the **grammar** exactly (`Grammar.random(seed=0)` is deterministic) but not the **trained weights** bit-for-bit. Every later pass (`results/refrun/`, `results/sweep/`, `results/arch/`, `results/depth4/`, `results/noncollapse/`) seeds model init from the master `--seed` and is fully deterministic.

9.1. Full pipeline in exact order

Everything needed to go from a clean checkout to a rebuilt `main.pdf` with every figure and table regenerated. Steps within a numbered stage are independent of each other; stages must run in the listed order because later stages read files earlier stages write. Total wall time is dominated by stages 3–6 (real MPS training): pass-1 (1 run), pass-2 (30 runs), pass-3 (99 runs), pass-4 (60 runs), pass-5 (27 runs) ≈ 216 training runs, each ≈ 25–45 s at the pinned arch (longer for larger `n_embd/n_layer/steps` cells in pass 3).

Stage 0 — sanity checks (seconds).

```
uv run pytest -q                # 10+ tests: BP == brute-force (<1e-6), weighted BP, grammar
                                # sampling
uv run python rhm.py            # self-test trace: sample length == s^L, entropy bloom 1.89->0.00
uv run ruff check                # lint
```

Stage 1 — pass-1 canonical run → `artifacts/` (not bit-exact, see caveat above).

```
uv run python train.py --steps 4000 --test-frac 0.1 --log-every 1000
uv run python analyze.py                # -> results/analysis.json
uv run python paper-typst/figures/scripts/root_reconstruction.py # -> results/
root_reconstruction.json
```

Stage 2 — pass-1 dense-loss reference run → `results/refrun/` (independent of stage 1; a separate seeded run used only for Figure 1).

```
uv run python -c "
from pathlib import Path
from types import SimpleNamespace
from train import train
args = SimpleNamespace(s=2, L=3, v=8, m=2, grammar_seed=0, seed=0,
    ambiguity=0.0, skew='none', n_layer=2, n_embd=128, n_head=4,
    lr=3e-3, steps=4000, batch_size=128, test_frac=0.1, n_probe=400, log_every=50)
train(args, out_dir=Path('results/refrun'))
"
```

Stage 3 — pass-1 figures (depend on stages 1–2: need `artifacts/`, `results/analysis.json`, `results/refrun/train_summary.json`).

```
uv run python paper-typst/figures/scripts/loss_curve.py      # @loss
uv run python paper-typst/figures/scripts/posterior_simplex.py # @simplex
uv run python paper-typst/figures/scripts/layer_position.py  # @layerpos
uv run python paper-typst/figures/scripts/latent_levels.py   # @latents
```



```
uv run python paper-typst/figures/scripts/blooming.py      # @blooming, @rootmatch
uv run python paper-typst/figures/scripts/steering.py      # @steering
```

Stage 4 — pass-2 grammar-generalization sweep → **results/sweep/** (30 runs; independent of stages 1–3, needs only sweep.py/analyze.py/train.py).

```
for g in $(seq 0 9); do for s in 0 1 2; do
  uv run python sweep.py --no-wandb --out-root results/sweep --grammar $g --seed $s
done; done
uv run python paper-typst/figures/scripts/grammar_sweep.py
# -> sweep_level_r2.png, sweep_blooming.png, sweep_steering.png, sweep_sanity.csv
```

Stage 5 — pass-3 architecture sweep → **results/arch/** (99 runs; independent of stages 1–4).

```
uv run python run_arch.py          # optionally --dry-run first to preview the 99-cell plan
uv run python paper-typst/figures/scripts/arch_sweep.py
# -> arch_marginal_r2.png, arch_depth_accum.png, arch_lossfit.png, arch_sanity.csv
```

Stage 6 — pass-4 depth-4 sweep → **results/depth4/** (60 runs; independent of stages 1–5).

```
uv run python run_depth4.py        # optionally --dry-run first
uv run python paper-typst/figures/scripts/depth4_sweep.py
# -> depth4_level_r2.png, depth4_blooming.png, depth4_layer_r2.png,
#     depth4_steering.png, depth4_sanity.csv, depth4_prereg.json
```

Stage 7 — pass-5 non-collapse sweep → **results/noncollapse/** (27 runs; independent of stages 1–6, but stages 8–9 depend on it).

```
uv run python run_noncollapse.py   # optionally --dry-run first
uv run python paper-typst/figures/scripts/noncollapse.py
# -> noncollapse_curve.png, noncollapse_heatmap.png, noncollapse_attractor.png,
#     noncollapse_sanity.csv, noncollapse_prereg.json
```

Stage 8 — grammar-as-graph figures (depend on stage 1 for the base grammar, and on stage 7 for the two pinned skew/ambiguity example dirs — must run **after** run_noncollapse.py, since its `__main__` regenerates all three PNGs in one process and errors if the noncollapse dirs don’t exist yet).

```
uv run python paper-typst/figures/scripts/ruletable_graph.py
# -> ruletable_graph.png, ruletable_graph_skew.png, ruletable_graph_amb.png
uv run python paper-typst/figures/scripts/ruletable_tables.py
# -> ruletable_skew.typ, ruletable_amb.typ (included verbatim by main.typ)
```

Stage 9 — grammar non-isomorphism check (verifies the pass-2 sweep’s 10 grammars are pairwise non-isomorphic; can run any time after stage 4).

```
uv run python grammar_iso.py
```

Stage 10 — compile the paper (depends on every stage above having produced its PNGs/CSVs/.typ includes under paper-typst/figures/).

```
ninja
```

Parallelization note. Stages 4, 5, 6, and 7 (the four sweeps) don’t read each other’s outputs and can run concurrently given the wall-clock budget to run multiple MPS training processes at once; stage 8 must still wait for both 1 and 7 to finish, and stage 10 must wait for all figure-producing stages.

9.2. Per-number and per-figure provenance

Abstract.

Number	File · key	Regenerate
≈399k parameters	artifacts/train_summary.json → n_params (398848)	Stage 1
1024 equiprobable trees	derived: $v \cdot m^{d-1} = 8 \cdot 2^7$	arithmetic, not a file
$< 10^{-6}$ BP vs brute-force	pytest (10 tests)	Stage 0
0.88 nats test CE / 89% gap closed	results/refrun/train_summary.json → final_test_loss, and $(u - f)/(u - b)$	Stages 1–3
$R^2 = 0.38$ root / ≈ 0 shuffled	results/analysis.json → latent_level_r2.root_L0, layer_r2_shuffled[-1]	Stage 3
−0.00 → 0.15 → 0.38 layer accumulation	results/analysis.json → layer_r2	Stage 3
up to 0.66 deep latents	results/analysis.json → latent_level_r2	Stage 3
−0.73 → −8.5 steering	results/analysis.json → steering.true_lp_steer_wrong	Stage 3
three of four predictions held	PREREGISTRATION.md scored by hand against results/analysis.json	Stages 1, 3
30-run sweep, 10 grammars × 3 seeds	figures/sweep_sanity.csv (30 rows)	Stage 4
99-run architecture sweep	figures/arch_sanity.csv (99 rows)	Stage 5

Experimental design — grammar and ground truth posterior. The frozen rule table (Table 1) and “1024 equiprobable trees” are read directly off artifacts/grammar.npz (rules_0/1/2, probs_0/1/2 if present). To regenerate the **graph** version instead of the text table: `uv run python paper-typst/figures/scripts/ruletable_graph.py` (Stage 8; reads artifacts/grammar.npz via `analyze.load_grammar`, writes figures/ruletable_graph.png). “ $< 10^{-6}$ across 10 passing tests” (BP vs brute-force): `uv run pytest -q` (Stage 0). “entropy 1.89 nats at $k = 1$ falling to 0.00 by $k = 3$ ” (the self-test preview): `uv run python rhm.py` (Stage 0; module self-test prints exactly this trace, see EXECUTION_OUTPUT.md § 1). “0.725 nats” Bayes floor / “2.079” uniform baseline: artifacts/train_summary.json → bayes_optimal_mean, uniform_baseline (Stage 1; computed inside `train.train()` from the exact grammar, independent of the trained model).

Model and training (Figure 1). Currently-informative file: results/refrun/train_summary.json → uniform_baseline, bayes_optimal_mean, final_test_loss, n_params, n_train, n_test, config{n_layer,n_embd,n_head}, loss_history (list of [step, train_minibatch_CE, held_out_test_CE], ≈81 points at log_every=50). This is the file the figure and the “0.880 nats / 89%” text both cite — **not** artifacts/train_summary.json, which has no loss_history (pass-1 predates that field). artifacts/train_summary.json holds the original pass-1 numbers (final_test_loss=0.8911, n_params=398848, n_train=922, n_test=102), underlying the abstract’s “≈399k parameters” and the EXECUTION_OUTPUT.md § 3 trace, but Fig. 1’s curve and caption numbers come from results/refrun/ (Stage 2). Figure: Stage 3, loss_curve.py.

Linear probe (Figure 2). Data: artifacts/probe_data.npz (beliefs $N \times 8 \times 8$, hidden $N \times 3 \times 8 \times 128$ — index [:,2] is the final-layer residual) for the geometry panel; results/analysis.json → latent_level_r2.root_L0 (0.38) and layer_r2_shuffled[-1] (−0.085) for the cited R^2 numbers. Figure: Stage 3, posterior_simplex.py (fits its own in-sample probe for the plotted geometry — the annotated “ $R^2 \approx 0.38$ ” is the held-out score from results/analysis.json, a separate fit; see the script’s header comment).

Layer/position (Figure 3). Data: results/analysis.json → layer_r2 (3 elements: embedding, after block 1, after block 2), layer_r2_shuffled, heatmap_layer_position_r2 (3×8 matrix). Figure: Stage 3, layer_position.py.

Latent hierarchy (Figure 4). Data: `results/analysis.json` → `latent_level_r2` (keys `root_L0`, `mid_L1a`, `mid_L1b`, `low_L2a..d`). Figure: Stage 3, `latent_levels.py`. The $L = 4$ inverted-U preview (`depth4_level_r2.png`, Figure 5) comes from the depth-4 sweep, Stage 6.

Blooming (Figure 6, Figure 21). Data: `artifacts/probe_data.npz` (`hidden[:,2]`, `beliefs`, `roots`). Radius numbers ($0.17 \rightarrow 0.39$ readout, $0.17 \rightarrow 0.47$ true posterior) and root-match rate (0.47) are printed by the script itself at generation time (Stage 3, `blooming.py`, writes both `blooming.png` and `blooming_match.png`; stdout prints “radius-from-prior by pos” and “root match rate”). “raw residual PCA — correlation ≈ 0.03 against ≈ 0.81 ” (the non-blooming control) is **not** persisted by any committed script — it exists only in prior session transcripts and the paper text (a known gap, below).

Causal steering (Figure 7). Data: `results/analysis.json` → `steering{alphas, patch_layer, true_lp_steer_true, true_lp_steer_wrong, target_mass_steer_true, target_mass_steer_wrong}`, computed by `analyze.causal_steering()` inside Stage 1’s `analyze.py` (patches layer-1 residual by class-mean difference, $\alpha \in \{0, 0.5, 1, 2, 4\}$). Figure: Stage 3, `steering.py`.

Pre-registration scorecard (Section 3.6). `PREREGISTRATION.md` records the four predictions, committed before training (honor-code rule); Table 2 scores them by hand against `results/analysis.json` — there is no separate `pass1_prereg.json`. Unlike passes 4–5, this scoring is manual prose, not a script-emitted JSON.

Non-collapsing belief geometry (Figure 8, Figure 9, Figure 10). Data: `results/noncollapse/*/{config.json, analysis.json}` — 27 run dirs ($3 \text{ skew} \times 3 \text{ ambiguity} \times 3 \text{ rule-table draws}$), pinned arch. Per-run `analysis.json` additionally carries a `noncollapse{ambiguity, skew, root_entropy_full_context, reachable_eff_dim, reachable_hull_area}` block (populated only when `g.ambiguity > 0` or `g.skew != 'none'`). Regeneration: Stage 7 (`run_noncollapse.py`, then `noncollapse.py` → figures, `noncollapse_sanity.csv`, `noncollapse_prereg.json`). “ 2.5×10^{-16} ” BP/brute-force agreement under both knobs: Stage 0’s pytest suite (weighted sum-product tests added alongside the ambiguity/skew knobs). The two example grammars in Appendix 4 are the pinned dirs `results/noncollapse/skhigh_am0_g00_L2_d128_h4_t4000_s0` and `results/noncollapse/sknone_am0.6_g00_L2_d128_h4_t4000_s0`, hardcoded in `ruletable_graph.py`’s and `ruletable_tables.py`’s `__main__` blocks (Stage 8).

Generalization across grammars (Figure 11, Figure 12, Figure 13, Table 5). Data: `results/sweep/g{00..09}_L2_d128_h4_t4000_s{0,1,2}/{config.json, analysis.json}` — 30 runs; figures/`sweep_sanity.csv` columns

`grammar, seed, n_layer, n_embd, n_head, test_ce, loss_gap_closed, root_r2, deepest_r2`. There is no dedicated `run_sweep.py` wrapper (unlike passes 3–5), so Stage 4’s 30-cell reproduction is a shell loop over `sweep.py` directly. Figures/table: `grammar_sweep.py` (globs `results/sweep/g*`). “ 0.89 ± 0.03 ” loss-gap-closed, “ 0.35 ± 0.07 ” root R^2 family mean, “ 6.9 ± 0.5 nats” steering collapse are printed to stdout by `grammar_sweep.py`’s `table_sanity()/fig_steering()`; mechanically reproducible via pandas:

```
d=pd.read_csv("figures/sweep_sanity.csv");
d[["loss_gap_closed", "root_r2"]].agg(["mean", "std"]). Grammar non-isomorphism check: Stage 9,
grammar_iso.py.
```

Architecture dependence (Figure 14, Figure 15, Figure 16, Table 6). Data: `results/arch/g{00,01,02}_L{1..4}_d{16,64,128,256}_h{1,2,4,8}_t{4000,16000}_s{0,1,2}/` — 99 runs (11 one-axis-at-a-time configs $\times$ 3 grammars $\times$ 3 seeds); figures/`arch_sanity.csv` columns

`grammar, seed, n_layer, n_embd, n_head, steps, test_ce, loss_gap_closed, root_r2, deepest_r2`.

Regeneration: Stage 5. All marginal-effect numbers and the `corr(loss_gap_closed,  $R^2$ )` values are printed to stdout by `arch_sweep.py`’s `fig_marginal()/fig_lossfit()`.

Going deeper: $L = 4$ (Figure 17, Figure 18, Figure 19, Figure 20, Table 7). Data: `results/depth4/g{00..09}_L{2,3}_d128_h4_t4000_s{0,1,2}/` — 60 runs; figures/`depth4_sanity.csv` columns `grammar, seed, n_layer, test_ce, loss_gap_closed, root_r2, deepest_r2` (`deepest_r2` = mean over the eight L_3 leaf-parent nodes). Regeneration: Stage 6. `PREREGISTRATION_L4.md` is scored against figures/`depth4_prereg.json` (`level_means`, `root_r2_mean_n2/n3`, `ladder_monotone_fraction`, `steering_collapse_mean/sd`, printed by `depth4_sweep.py`’s `if __name__` block).

Root reconstruction diagnostics (Figure 21). Data: `results/root_reconstruction.json` (`meta{N,positions,root_classes,chance_rate,chance_percent}`, `all_positions`, `per_position[]`, `all_but_last`, `full_sequence`; each a `{matches,total,rate,percent}` pair for readout vs `exact_posterior_map`). Regeneration: Stage 1, `root_reconstruction.py` (reads `artifacts/probe_data.npz`; note `blooming_match.png` itself is written by `blooming.py`, Stage 3, not this script).

Rule table as a graph and example grammars (Figure 22, Figure 23, Figure 24). Regeneration: Stage 8, `ruletable_graph.py` (regenerates all three graph variants in one run: base from `artifacts/`, skew and ambiguous variants from the two pinned `results/noncollapse/...` dirs) and `ruletable_tables.py` (writes the two `.typ` table includes).

Per-run sanity tables (Table 5, Table 6, Table 7, Table 8). All four `*_sanity.csv` files are read directly by this document via `csv(...)` — regenerating the CSV in place does not require touching the Typst source. Regeneration commands are the four aggregator scripts (Stages 4–7): `grammar_sweep.py` (30 rows), `arch_sweep.py` (99 rows), `depth4_sweep.py` (60 rows), `noncollapse.py` (27 rows).

Known gaps (not mechanically regenerable as of this writing).

- Raw-residual-PCA “correlation ≈ 0.03 vs ≈ 0.81 ” (the blooming-section honesty check) is not computed by any committed script; it exists only in prior session transcripts and the paper text.
- Pass 2 (`results/sweep/`) has no dedicated `run_*.py` wrapper analogous to passes 3–5, so its 30-cell reproduction is a shell loop over `sweep.py` rather than a single `uv run python run_sweep.py`.
- The pass-1 Table 2 scoring is hand-authored prose against `results/analysis.json`, not emitted by any script (unlike the $L = 4$ and non-collapse scorecards, which have `*_prereg.json`).